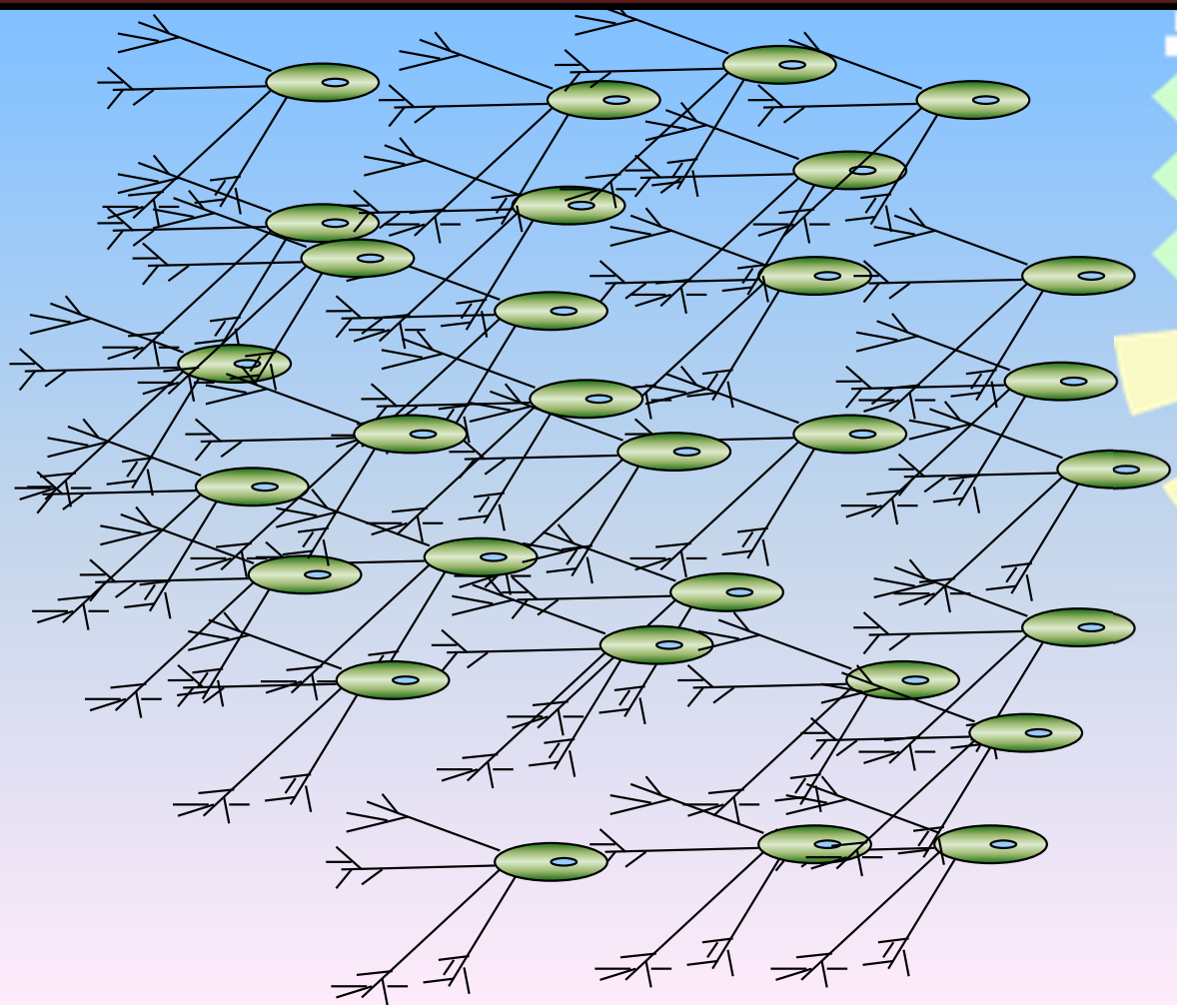
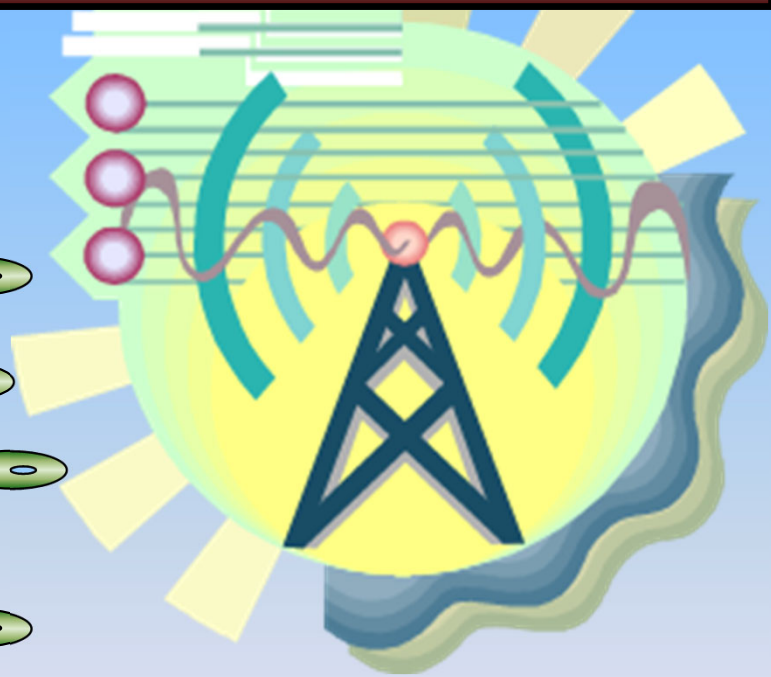
Neural Networks

Part 1





n.h.reyes@massey.ac.nz

Neural Networks

- Introduction to Multi-layer Feed-forward Network
- Evaluating a Feed-forward Network
- Training a Feed-forward Network
- Theory (Derivation of Formulas)
- Training Tips
- Encoding Inputs, Softmax units
- Performance Measure
- Applications, Tutorial Questions

Artificial Neural Networks

- Computational models made up of simple, highly interconnected neuron-like units which operate in some parallel and distributed mode to perform some common global task

- connectionist systems, PDP networks, neural computing devices, empirical learning systems, etc.



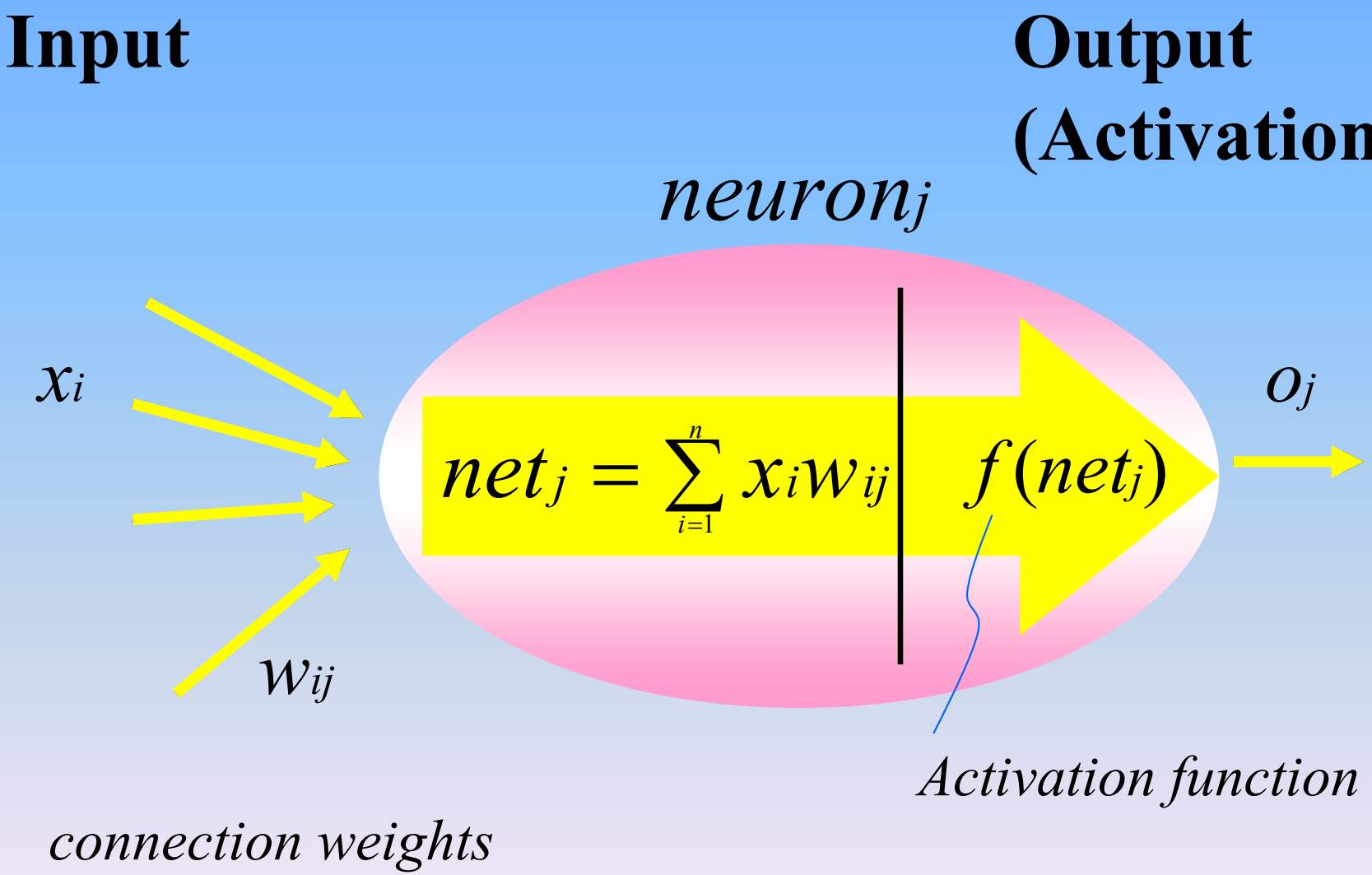
Artificial Neural Networks

- Neural nets are quantitative, numerical and don't require a knowledge engineer to extract expert information
- Neural networks are inductive programs; they take in a great amount of information all at once and then draw a conclusion.

Artificial Neural Networks

- Learning ability
- inherent parallelism
- distributed mode of operation
- simplicity of units' behavior
- absence of centralized control

the artificial neuron



Activation/ Squashing/Transfer Function

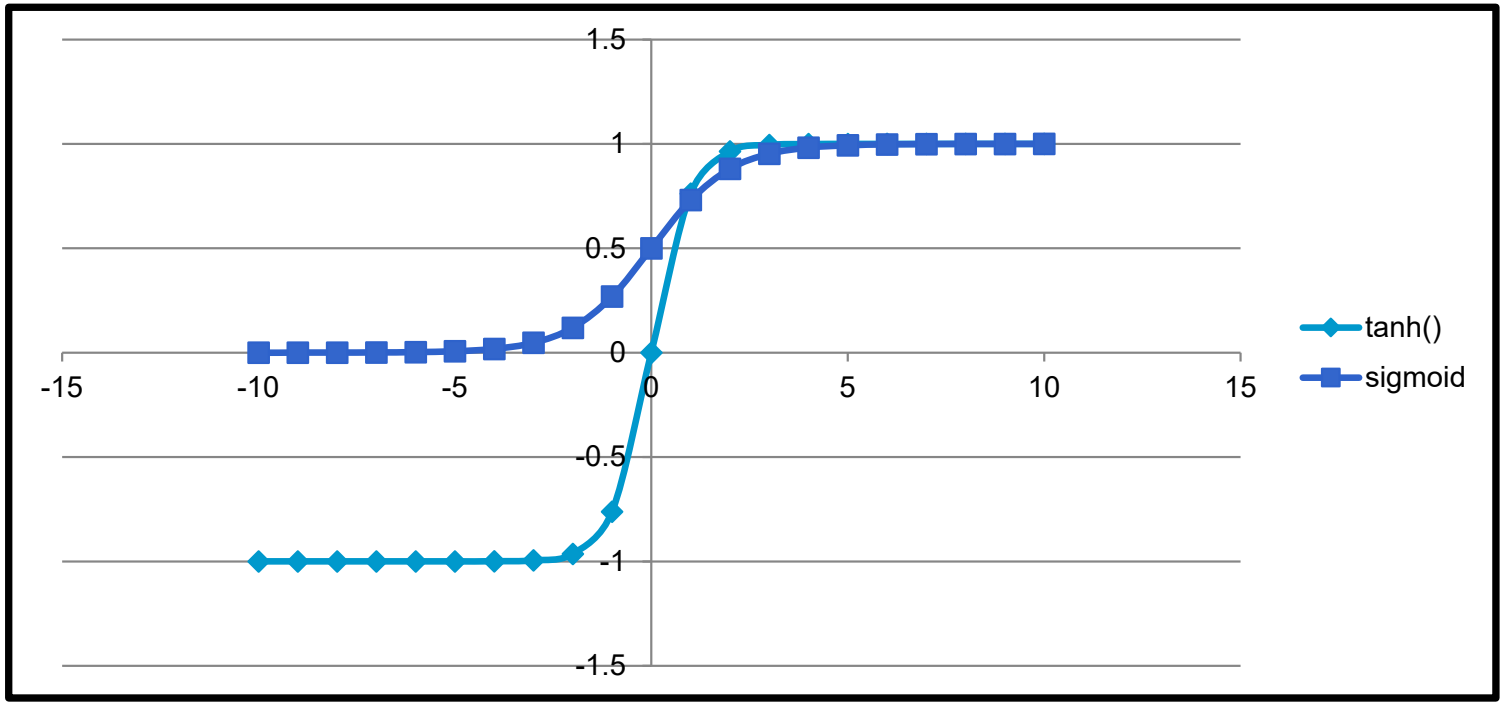
where

$$f(net_j) = \frac{1}{1+\exp^{(-netj)}}$$

Activation Functions

- Logistic function or Sigmoid function
- Hyperbolic tangent,
- ReLU, etc.

Activation Functions



• The **sigmoid function** varies between zero and one. This is inconvenient in some calculations as it cannot handle negative values.

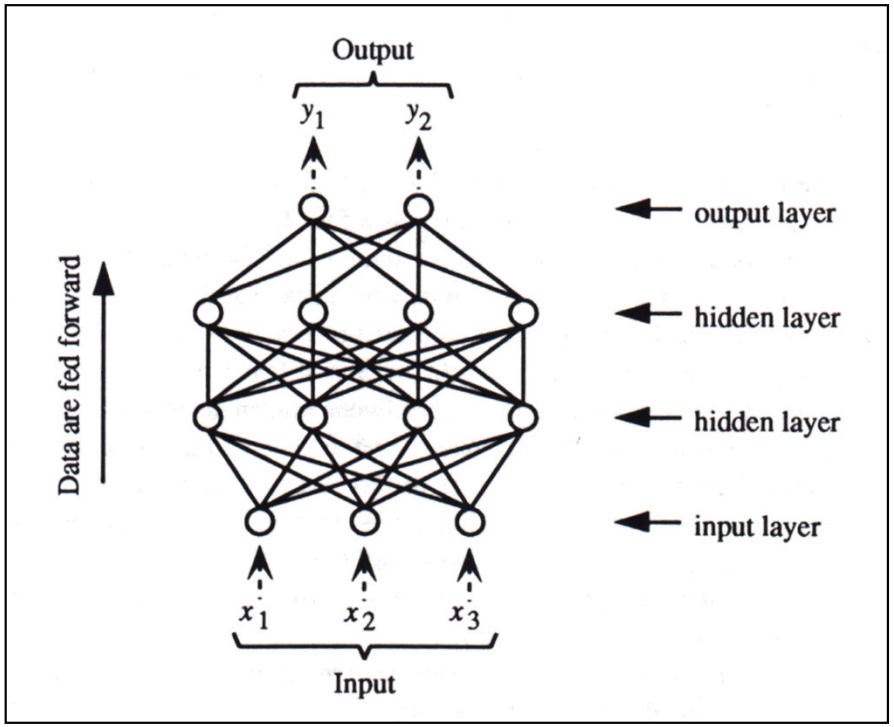
• It is sometimes more convenient to use the ***tanh()* function**, which has exactly the same shape but which swings between +/- 1.

Activation Functions and their derivatives

Activation Function	$g(a)$	$g'(a)$
Linear	$g(a) = a$	$g'(a) = 1$
Logistic	$g(a) = \frac{1}{1+e^{-a}}$	$g'(a) = g(a)(1-g(a))$
Hyperbolic-tangent	$g(a) = tanh(a)$	$g'(a) = sech^2(a) = 1-tanh^2(a)$
Squash	$g(a) = \frac{a}{1+\lvert a\rvert}$	$g'(a) = \frac{1}{(1+\lvert a\rvert)^2} = (1-\lvert g(a)\rvert)^2$

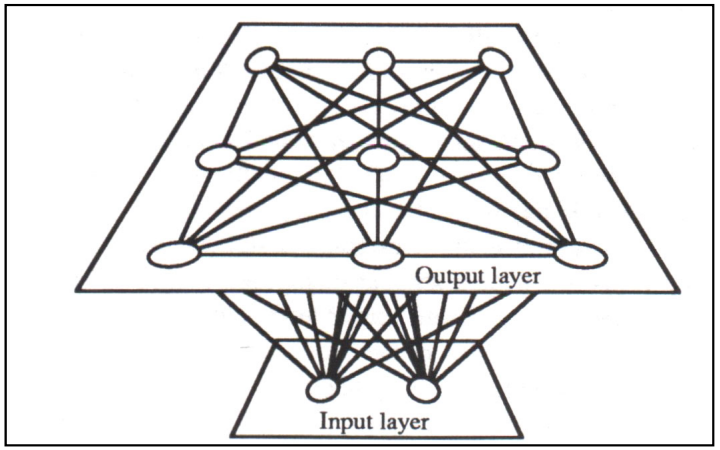
Bipolar sigmoidal $g(a) = \frac{2}{1+\exp(-a)} - 1$ $g'(a) = \frac{2\exp(-a)}{(1+\exp(-a))^2} = \frac{1}{2}(1-g(a))$

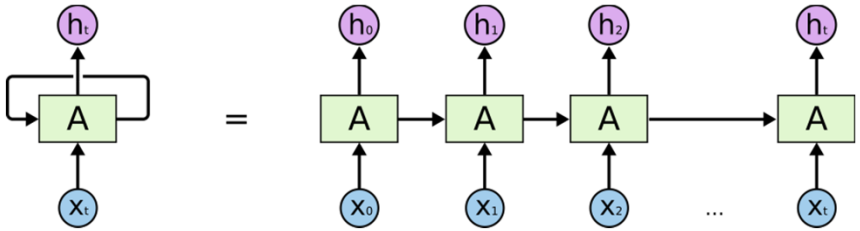
Various NN Architectures



Feed-forward Net

SOM





RNN (e.g. Long Short Term Memory networks)

Neural Network Models

- **Perceptron**
- Hopfield Networks
- Bi-Directional Associative Memory
- Self-Organizing Maps
- Neocognitron
- Adaptive Resonance Theory
- Boltzmann Machine
- Radial Basis Function Networks
- Cascade-Correlation Networks
- Reduced-Coulomb Energy Networks
- **Multi-layer Feed-forward Network**

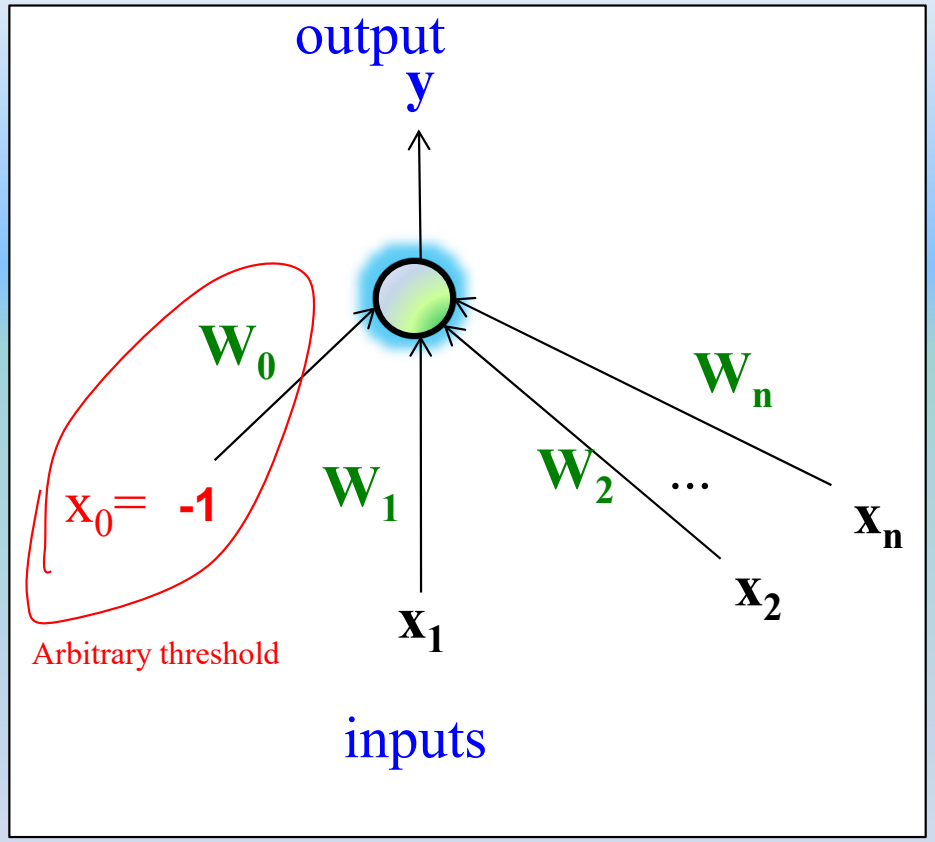


What have we learned previously?

- Single Perceptron Unit
- Multi-layer Perceptrons
- Introduction to Neural Networks



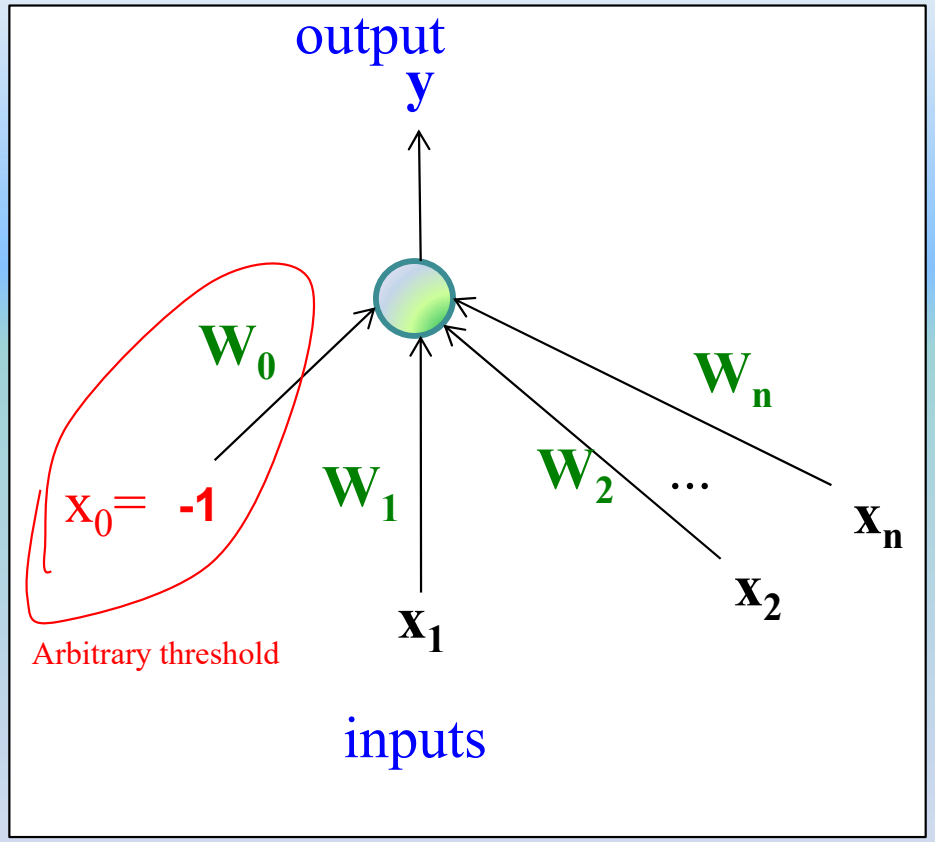
Linear Classifier Single Perceptron Unit



$$\theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Weighted input

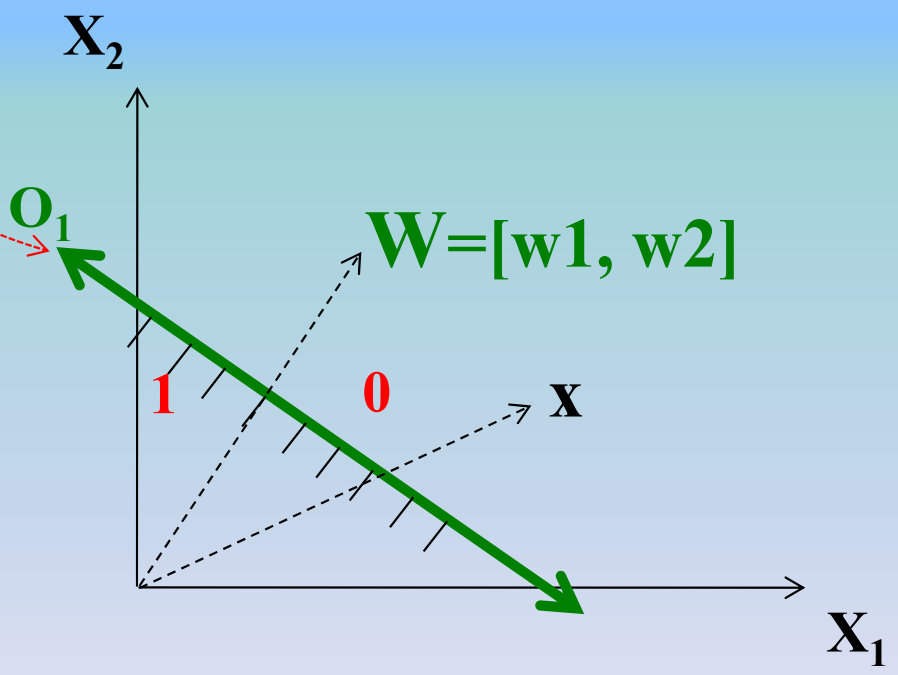
Linear Classifier Single Perceptron Unit



$$\theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

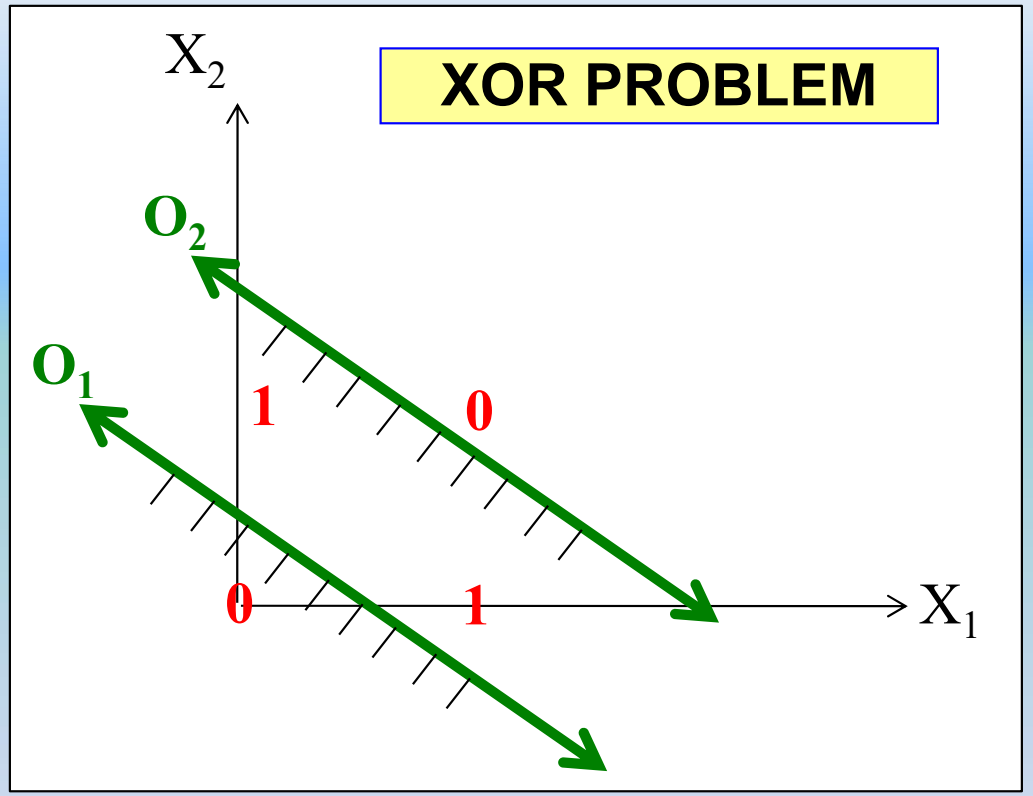
Weighted input

The decision boundary for a single perceptron is a **hyperplane** in the feature space.



Linear Classifier Single Perceptron Unit

Beyond Linear Separability

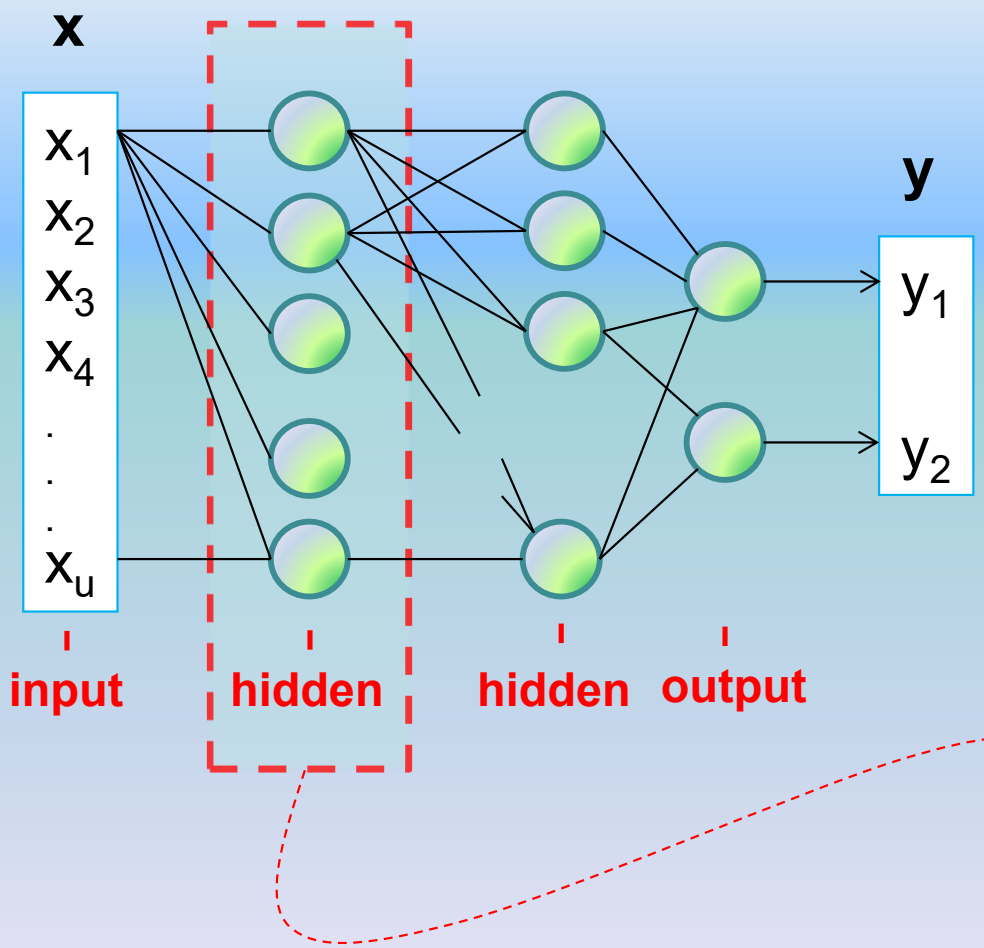


Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

What about if we consider more than one linear separator and combine their outputs; can we get a more powerful classifier?

Yes, since a single perceptron unit is so limited, a network of these units will be less limited.

Multi-Layer Perceptron



The introduction of “hidden” (not connected directly to the output) units into these networks make them much more powerful: they are no longer limited to linearly separable problems.

The lower layers transform the input problem into more tractable (linearly separable) problems for subsequent layers.

Multilayer Layer Perceptron

Any set of training points can be separated by a three-layer perceptron network (with sufficiently many units).

"Almost any" set of points are separable by a two-layer perceptron network (with lots of units).

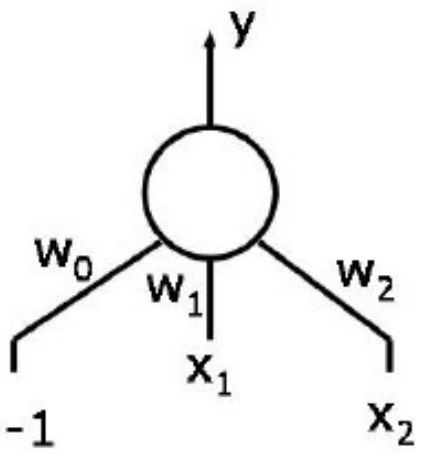
For example, (2^n)/n hidden units are needed to encode all boolean functions of n inputs.

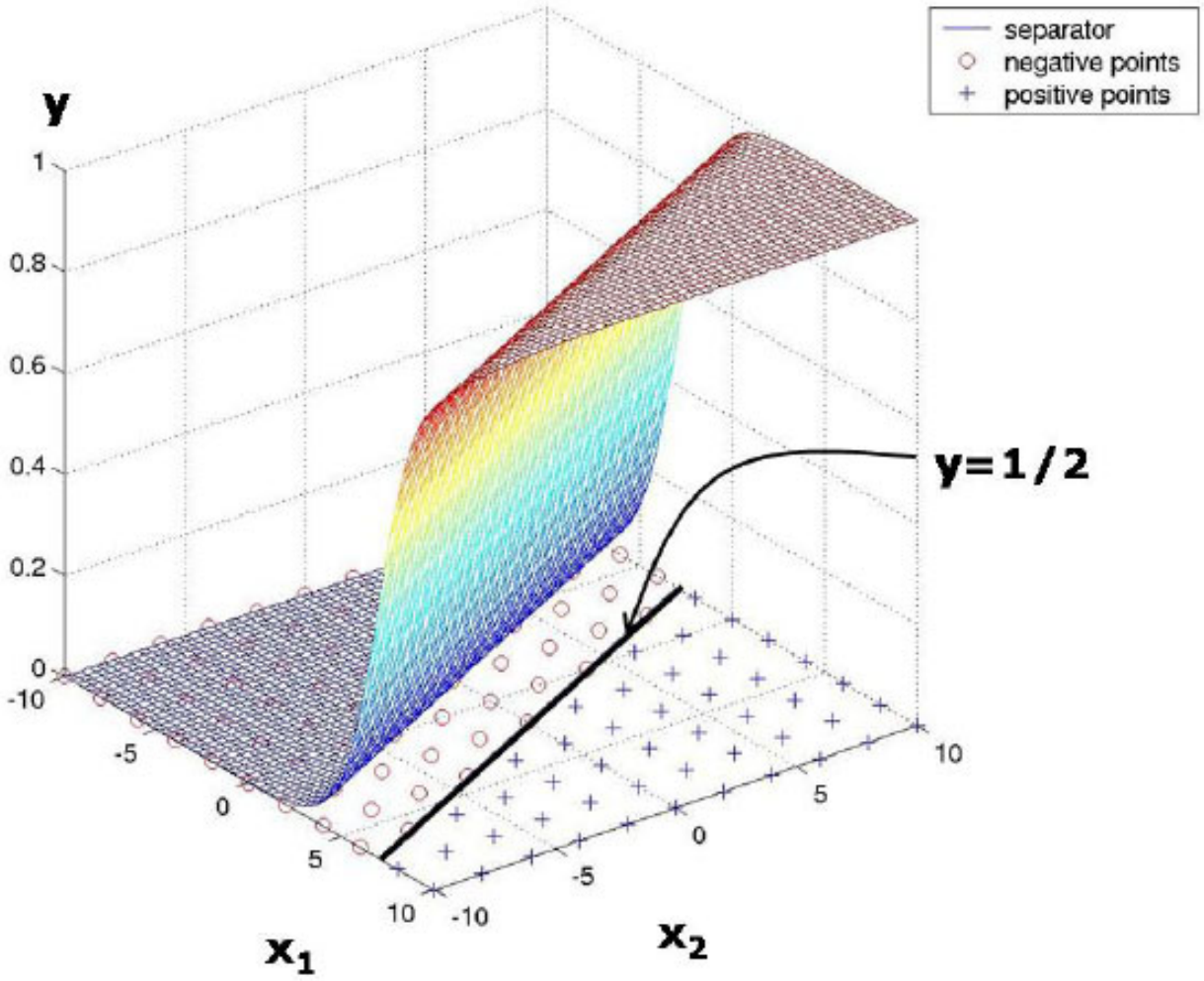
But no efficient learning rule is known.

Threshold function is not smooth.
We need a smooth function in lieu of the threshold function.

We need a smooth output (as a function of changes in the network weights) in order for us to use gradient descent.

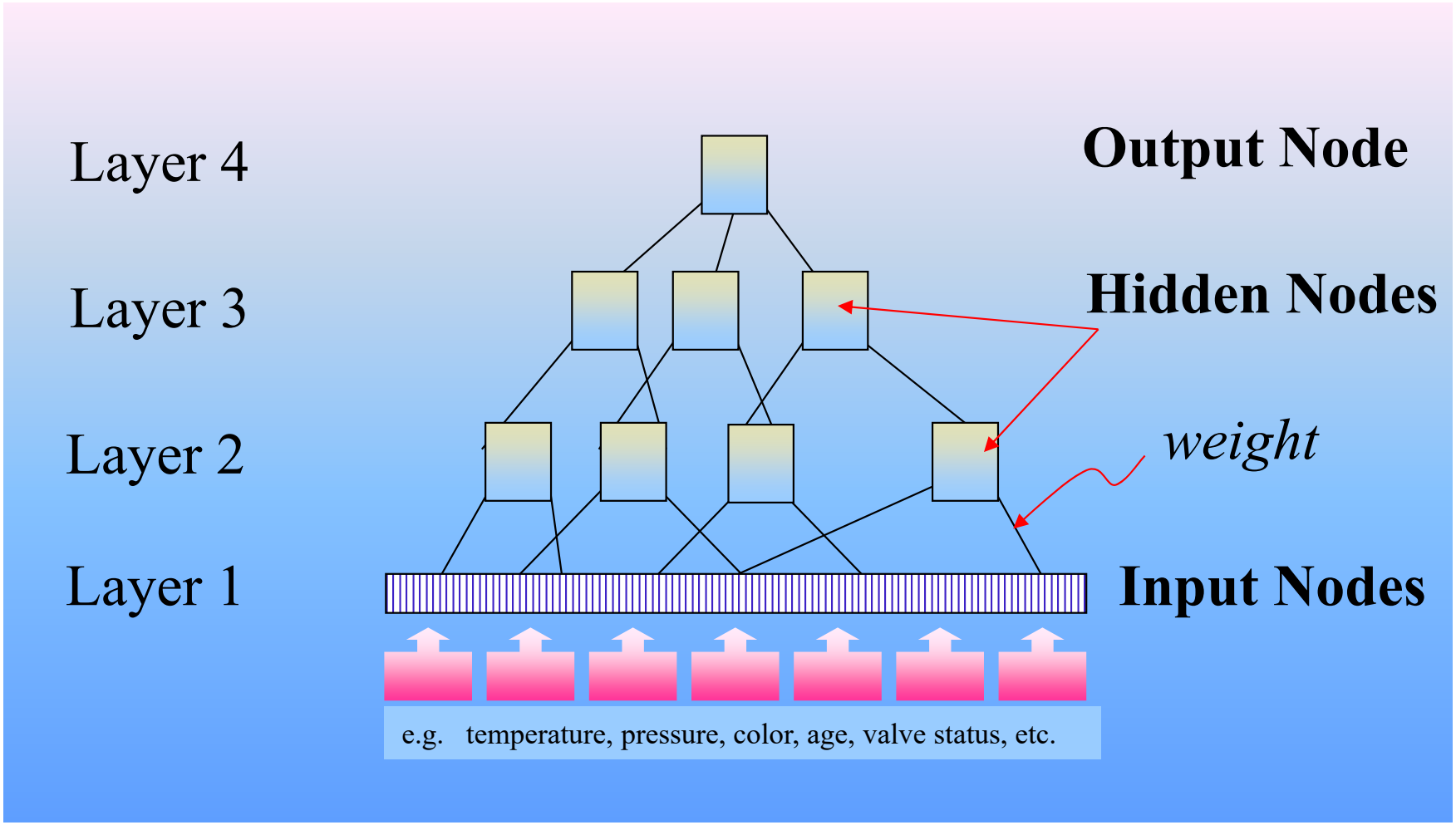
Sigmoid Unit





Neural Network Architecture

Multi-layer Feed-forward Network

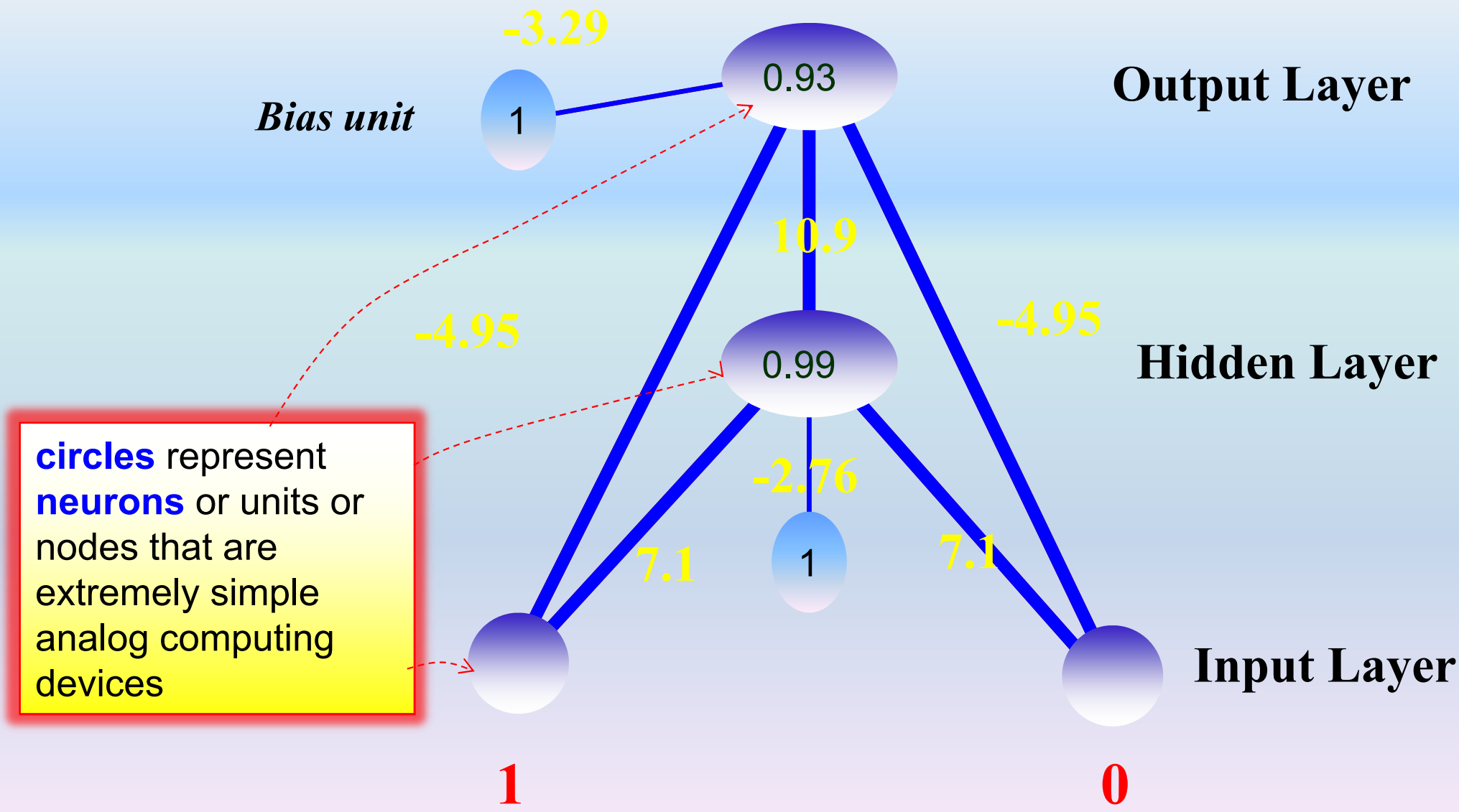


Example

Evaluating a Feed-forward Network

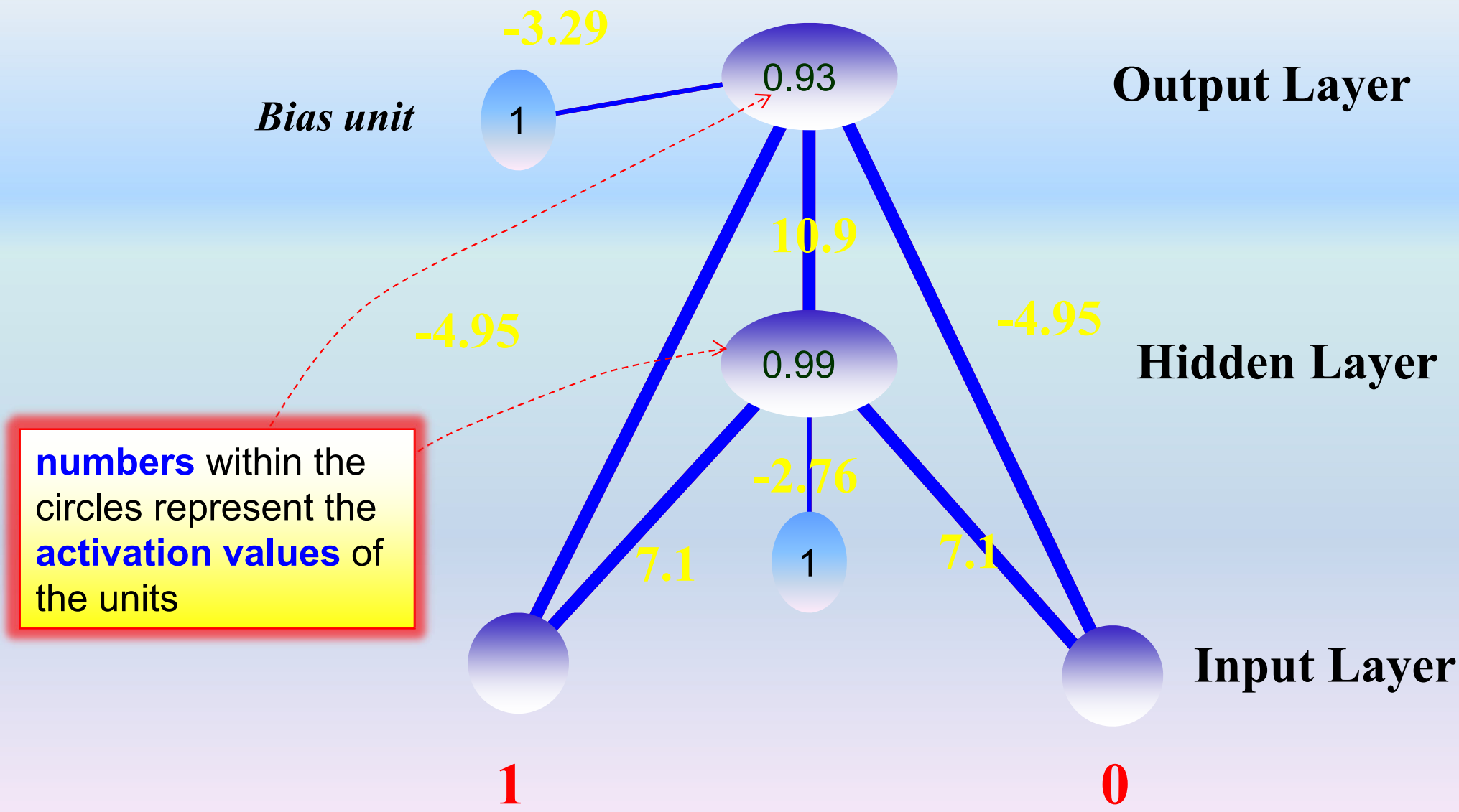
Multi-layer Feed-forward Network

Three-layer Network (2-1-1) for solving the XOR Problem

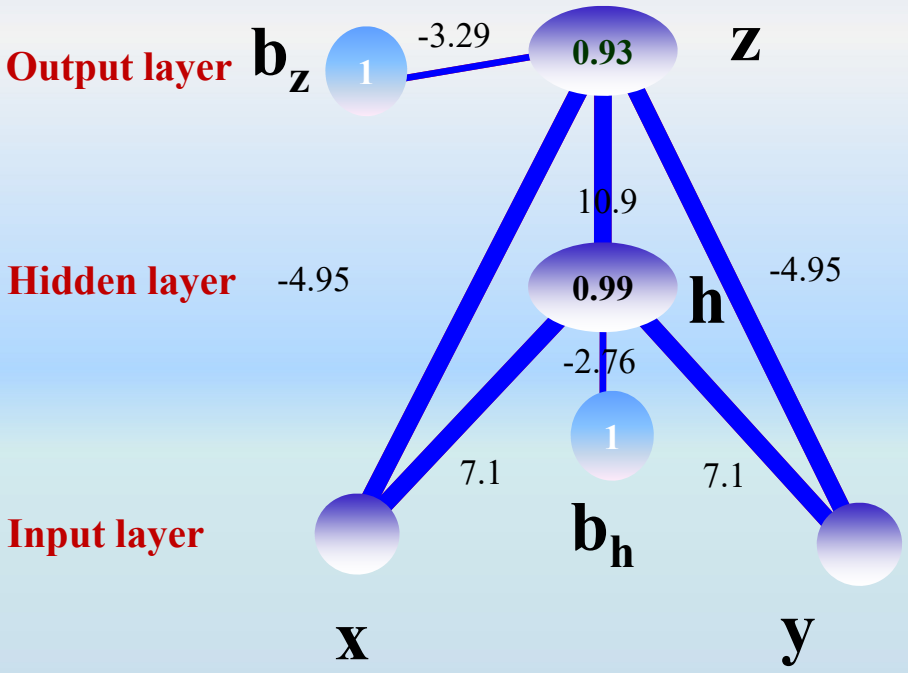


Multi-layer Feed-forward Network

Three-layer Network (2-1-1) for solving the XOR Problem

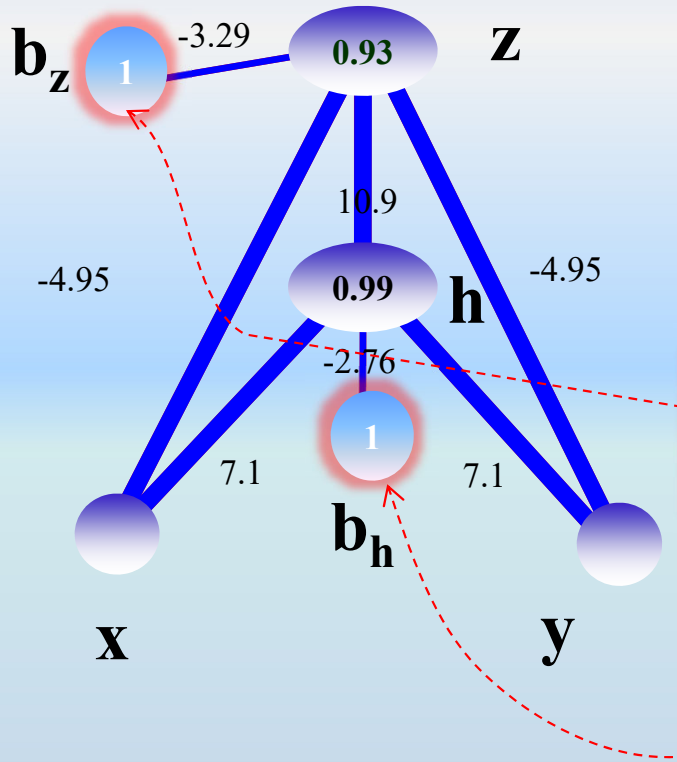


Multi-layer Feed-forward Network



There are three layers, the input layer that contains the values for x and y, a hidden layer that contains one node h, and an output unit that gives the value of the output value, z

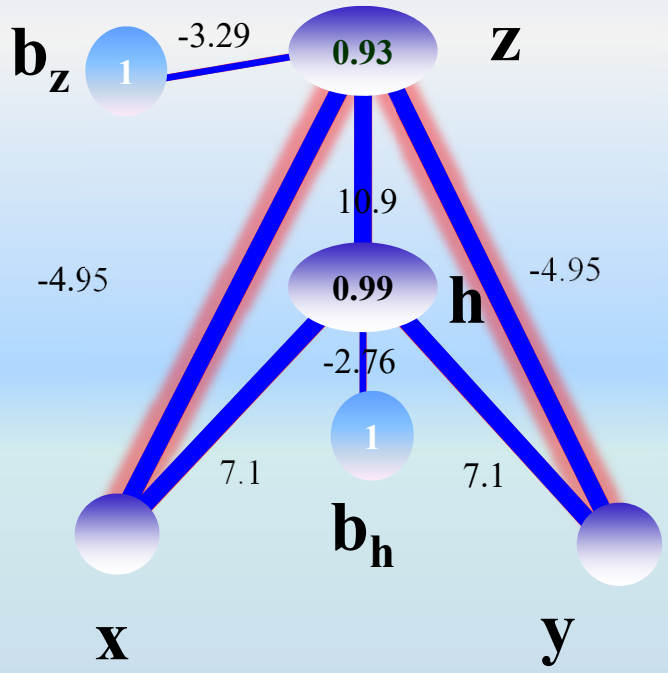
Multi-layer Feed-forward Network



There are three layers, the input layer that contains the values for x and y, a hidden layer that contains one node h, and an output unit that gives the value of the output value, z

There are two other units present called **bias units** whose values are always **1.0**

Multi-layer Feed-forward Network

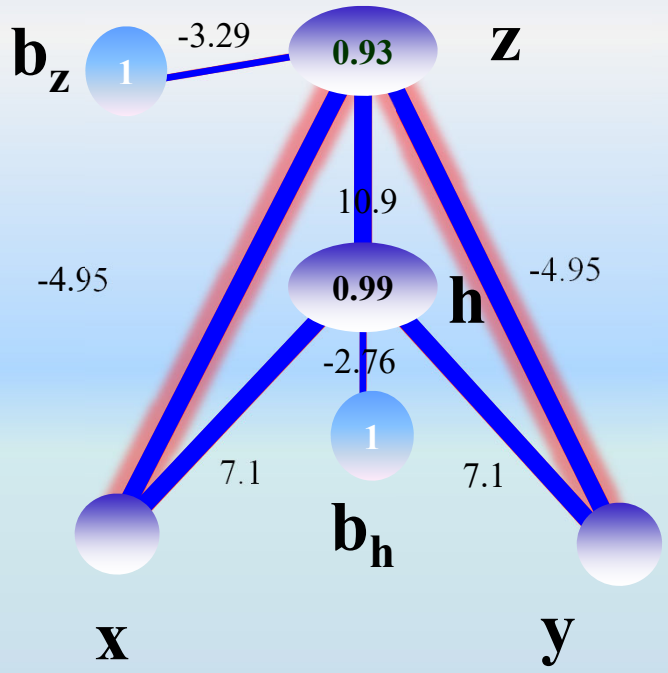


There are three layers, the input layer that contains the values for x and y, a hidden layer that contains one node h, and an output unit that gives the value of the output value, z

There are two other units present called bias units whose values are always 1.0

Much of the time feed-forward networks only have connections within adjacent layers; however, this one has two extra connections that go directly from the input units to the output unit.

Multi-layer Feed-forward Network



There are three layers, the input layer that contains the values for x and y, a hidden layer that contains one node h, and an output unit that gives the value of the output value, z

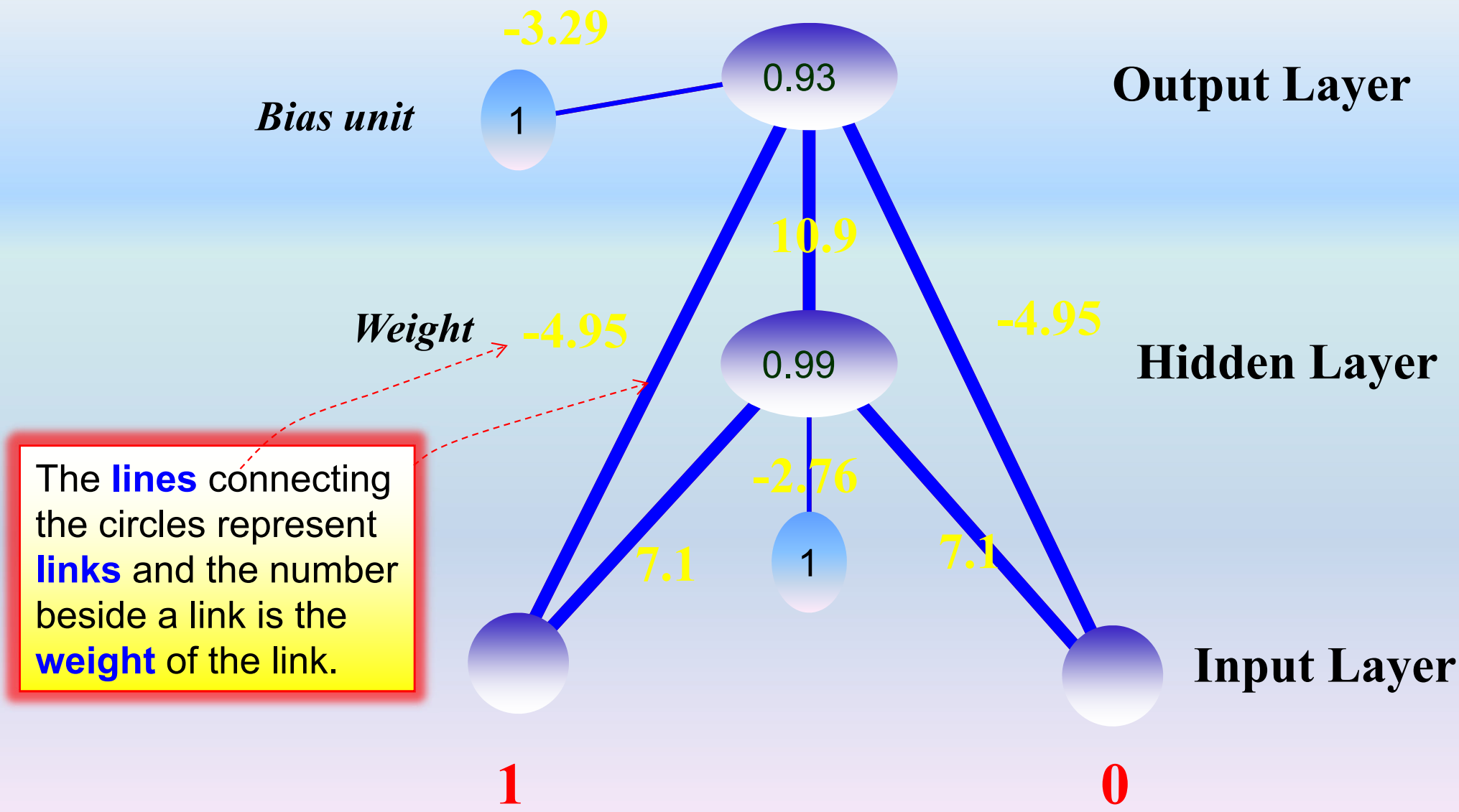
There are two other units present called bias units whose values are always 1.0

Much of the time feed-forward networks only have connections within adjacent layers; however, this one has two extra connections that go directly from the input units to the output unit.

In some problems, like XOR these extra input-output connections make training the network much faster.

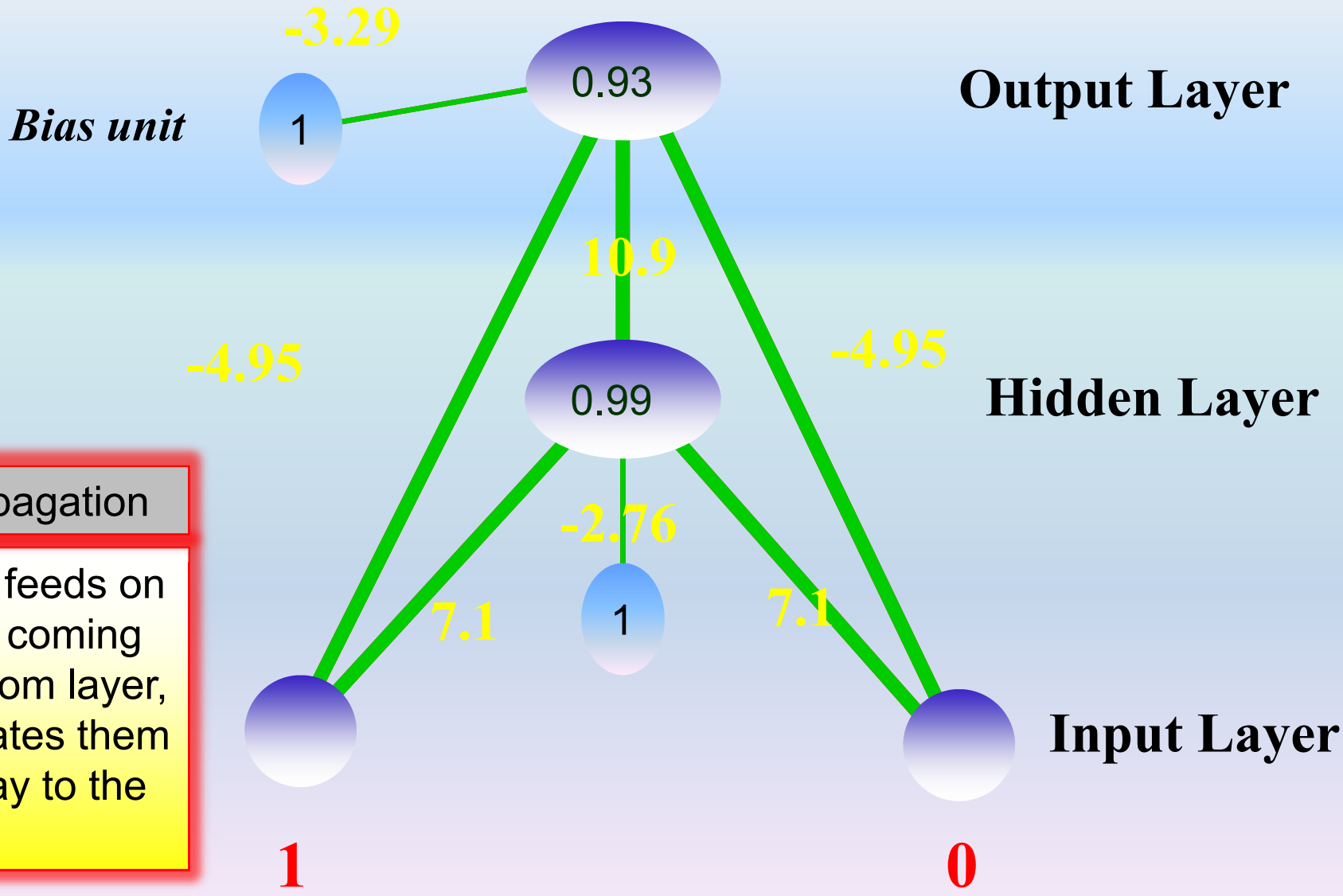
Multi-layer Feed-forward Network

Three-layered Network (2-1-1) for solving the XOR Problem



Multi-layer Feed-forward Network

Three-layered Network (2-1-1) for solving the XOR Problem

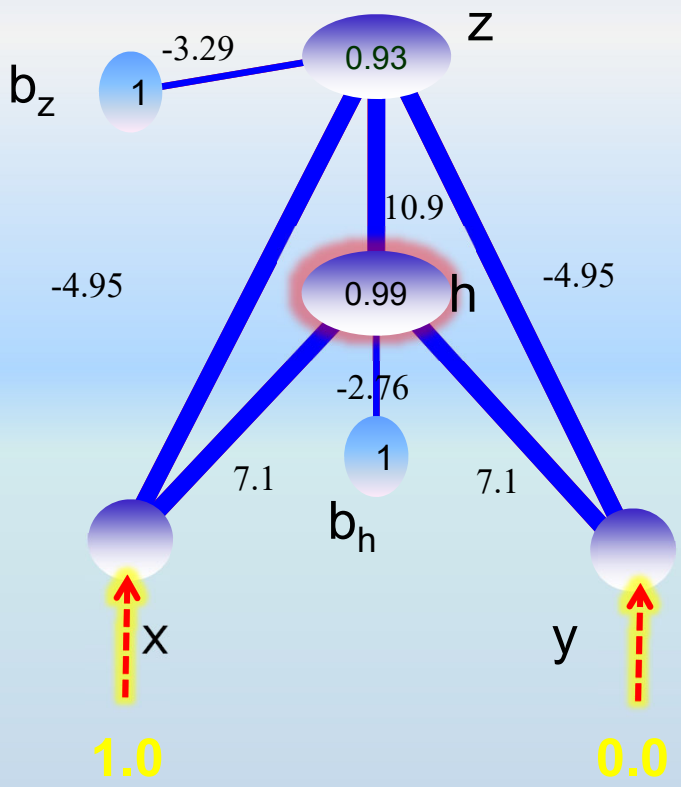


Forward propagation

The network feeds on input signals coming from the bottom layer, then propagates them up, all the way to the top.

Evaluating a network

Evaluating a Network



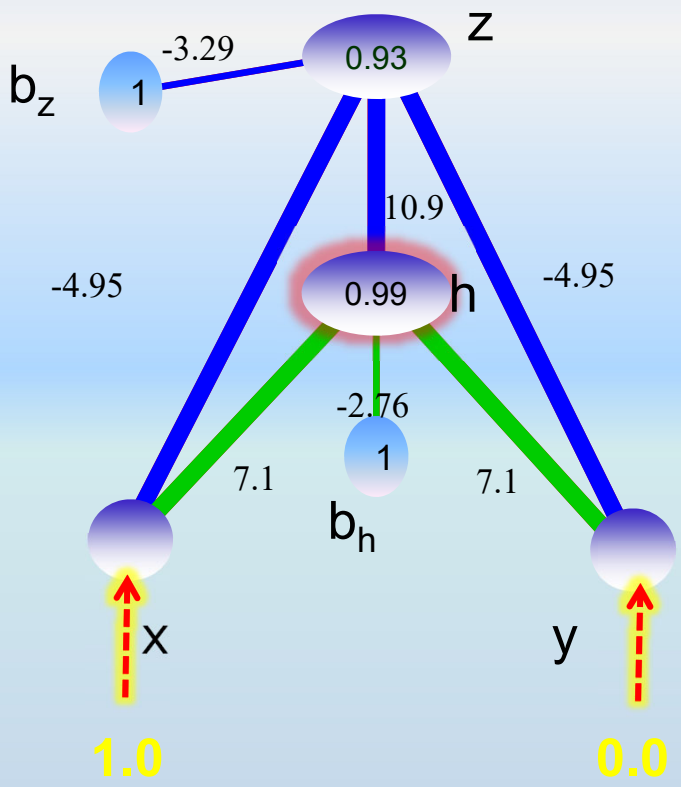
Example

Given a trained NN, and the following input values, calculate the output of the NN.

x = 1.0 and **y** = 0.0,

To compute the value of the output unit, z, we place values for x and y on the input layer units,
say **x** = 1.0 and **y** = 0.0, then propagate the signals up to the next succeeding layer.

Evaluating a Network (XOR Problem)



Example

Given a trained NN, and the following input values, calculate the output of the NN.

x = 1.0 and **y** = 0.0,

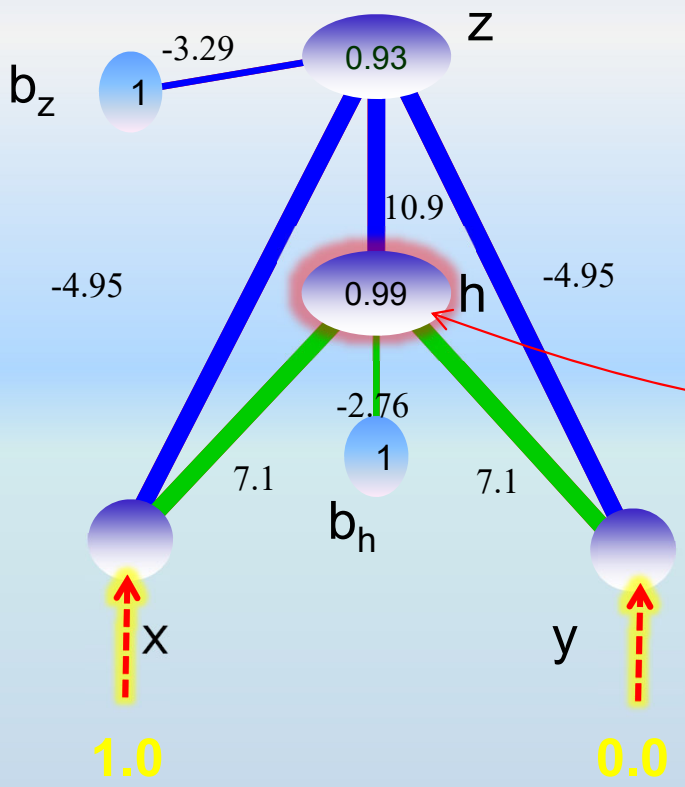
For the hidden node ***h***, find all lower level units connected to it.

For each of the connections, multiply the weight attached to the link by the value of the unit, then sum them all up. This sum is the net input of node ***h***.

Hidden Node h:

1.00	*	7.1	=7.1
1.00	*	-2.76	=-2.76
0.00	*	7.1	=0
		sum	4.34

Evaluating a Network (XOR Problem)



Example

Given a trained NN, and the following input values, calculate the output of the NN.

x = 1.0 and **y** = 0.0,

The output or activation value of node h is computed using a non-linear function:

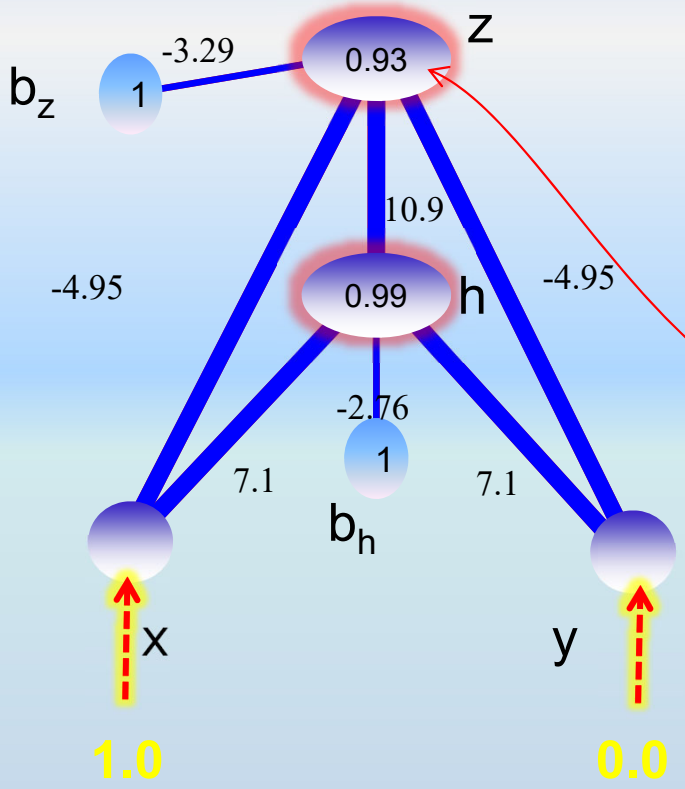
$$activationValue = \frac{1}{1+e^{-sum}}$$

Hidden Node h:

1.00	*	7.1	7.1
1.00	*	-2.76	-2.76
0.00	*	7.1	0
		sum	4.34

$$Node_h_{activation} = \frac{1}{1+e^{-4.34}} = 0.987131$$

Evaluating a Network (XOR Problem)



Example

Given a trained NN, and the following input values, calculate the output of the NN.

x = 1.0 and **y** = 0.0,

Continuing with the process, after propagating the signals toward output node **z**, we have:

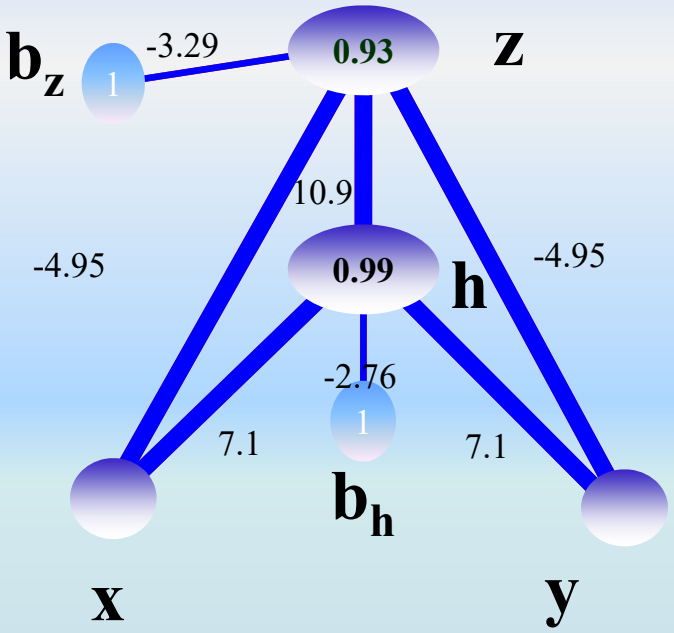
Hidden Node h:

1.00	*	7.1	7.1
1.00	*	-2.76	-2.76
0.00	*	7.1	0
		sum	4.34
		activation	0.987131

Output Node z

1.00	*	-4.95	-4.95
0.00	*	-4.95	0
0.99	*	10.9	10.791
1.00	*	-3.29	-3.29
		sum	2.551
		activation	0.93

Evaluating a Network (XOR Problem)



1.00	*	7.1	7.1
1.00	*	-2.76	-2.76
0.00	*	7.1	0
		sum	4.34
		v	0.987131

Output Node z

1.00	*	-4.95	-4.95
0.00	*	-4.95	0
0.99	*	10.9	10.791
1.00	*	-3.29	-3.29
		sum	2.551
		v	0.93

In some neural networks we might just leave the activation value of the unit to be **4.34**. In this case we would say that we are using the **linear activation function**, however backprop is at its best when this value is passed to certain types of **nonlinear functions**.

The most commonly used nonlinear function is:

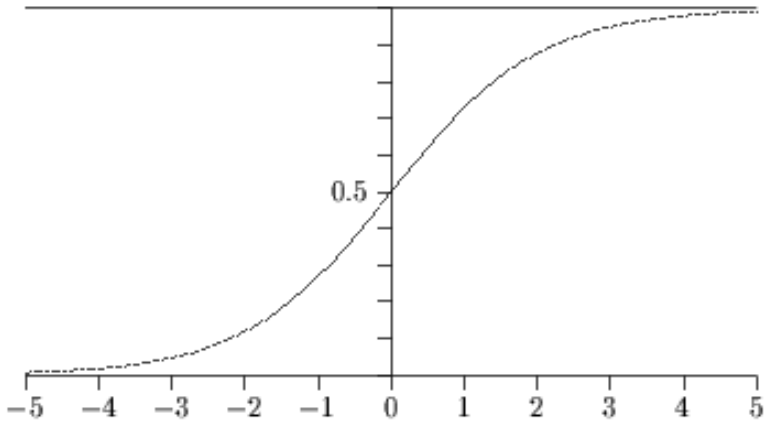
Standard Sigmoid:

$$v = \frac{1}{1 + e^{-s}}$$

where **s** is the sum of the inputs to the neuron and **v** is the value of the neuron. Thus, with **s** = 4.34, **v** = 0.99

Of course, 0.93 is not quite 1 but for this example it is close enough. When using this particular activation function for a problem where the output is supposed to be a 0 or 1, getting the output to within 0.1 of the target value is a very common standard.

Evaluating a Network



With this particular activation function it is actually somewhat hard to get very close to **1** or **0** because the function only approaches **1** and **0** as the input to the function approaches

∞ and $-\infty$

Standard Sigmoid:

$$v = \frac{1}{1 + e^{-s}}$$

Evaluating a Network

Computing the activation value of a Neuron

The formulas for computing the activation value for a neuron, **j** can be written more concisely as follows:

Let the **weight** between neuron **i** and neuron **j** be $\mathbf{w_{ij}}$. Let the net input to neuron **j** be $\mathbf{net_j}$

Let the activation value for neuron **j** be $\mathbf{O_j}$.

Let the **activation function** be the general function, **f**, then

$$net_j = \sum_{i=1,n} w_{ij} o_i$$

where **n** is the number of units feeding into unit **j** and

$$o_j = f(net_j)$$

Neural Networks

- [x] Introduction to Multi-layer Feed-forward Network
- [x] Evaluating a Feed-forward Network
- [ ] Training a Feed-forward Network
- [ ] Theory (Derivation of Formulas)
- [ ] Training Tips
- [ ] Encoding Inputs, Softmax units
- [ ] Performance Measure
- [ ] Applications, Tutorial Questions

Training a Network

Back propagation Algorithm

Backpropagation Training

On-line / Continuous Update : Method 1

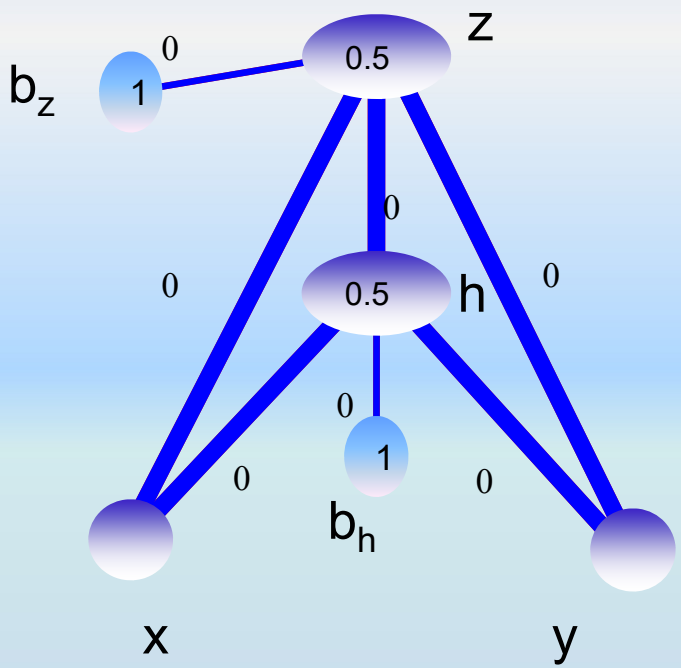
Iterative minimization of error over training set

1. Put one of the training patterns to be learned on the input units.
2. Find the values for the hidden unit and output unit.
3. Find out how large the error is on the output unit.
4. Use one of the back-propagation formulas to adjust the weights leading into the output unit.
5. Use another formula to find out errors for the hidden layer unit.
6. Adjust the weights leading into the hidden layer unit via another formula.
7. Repeat steps 1 thru 6 for the second, third patterns,…

Example: Training a network

Considering the same neural network example used for solving the XOR problem, let's examine how a neural network learns.

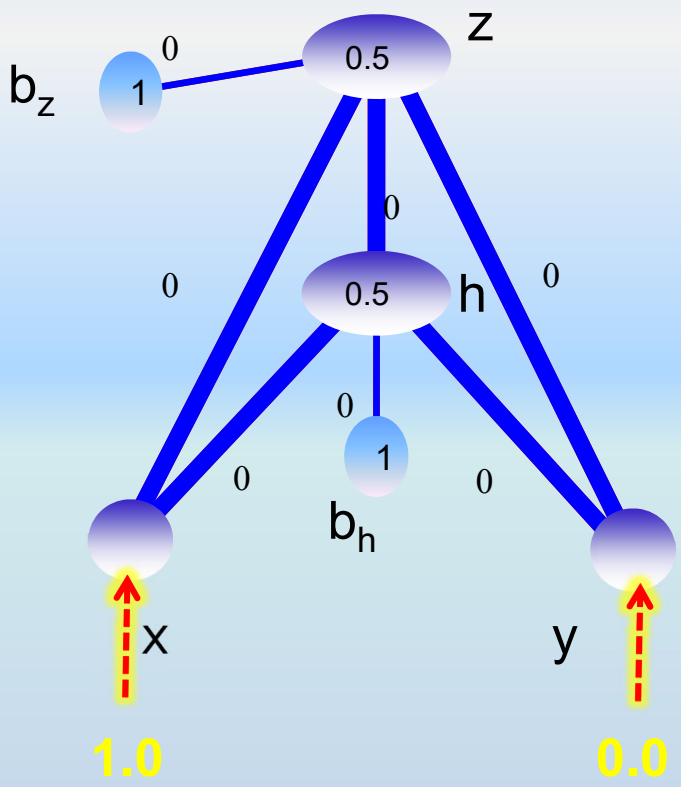
Training a Network



Firstly, give the network some initial weights.

For brevity of discussions, this example is using zero initial weights, but in practice, neural networks are initialised with randomly generated near-zero weights.

Training a Network



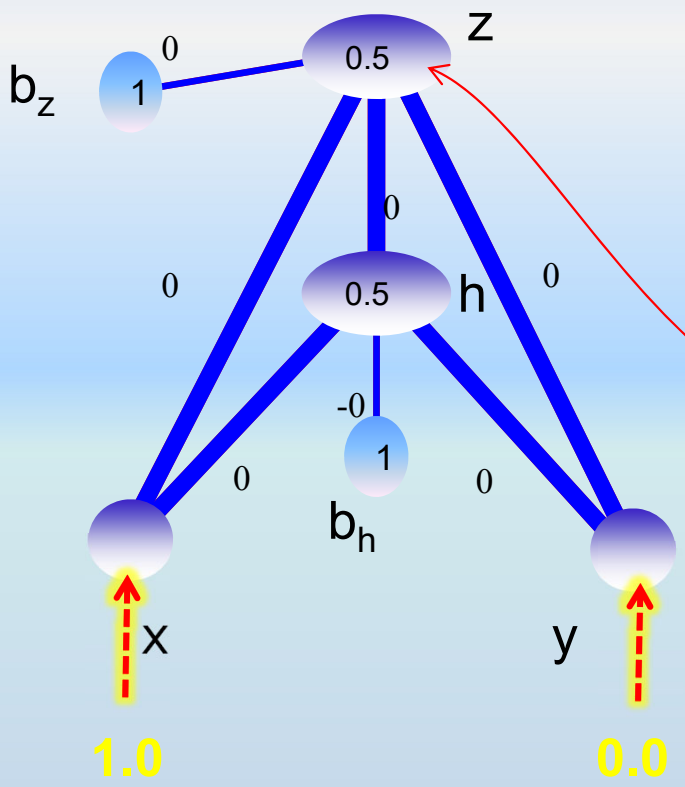
Randomly pick one training pattern from the training set, then feed it to the network.

e.g.
x = 1.0 and **y** = 0.0 , **desired output** = 1.0

TRAINING SET

x	y	desired
0	0	0
0	1	1
1	0	1
1	1	0

Training a Network



Randomly pick one training pattern from the training set, then feed it to the network.

e.g.
x = 1.0 and **y** = 0.0 , **desired output** = 1.0

Perform **forward propagation** until the activation value of the output neuron is found.

$$activationValue = \frac{1}{1+e^{-sum}} = \frac{1}{1+e^{0}} = \frac{1}{2}$$

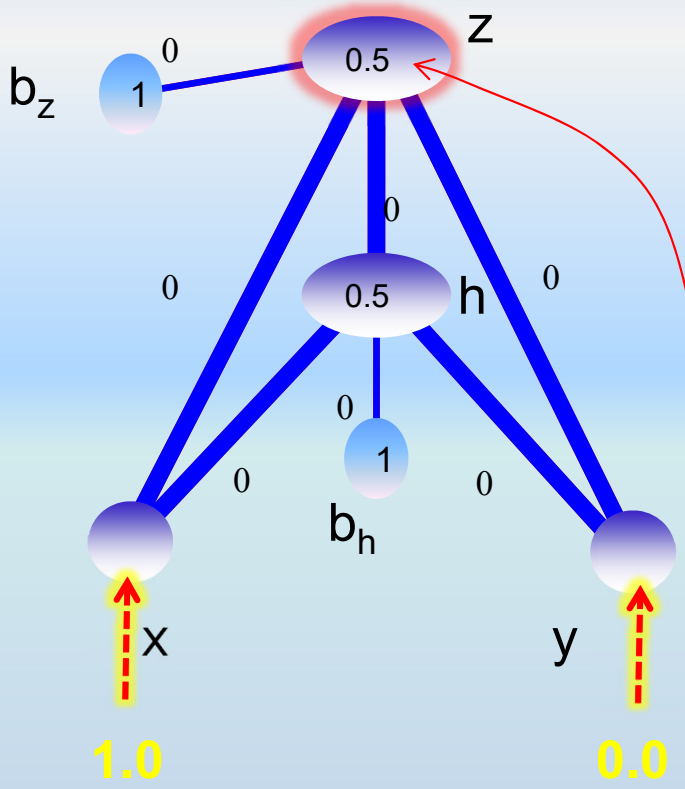
Hidden Node h:

1.00	*	0	0
1.00	*	0	0
0.00	*	0	0
		sum	0
		activation	0.5

Output Node z

1.00	*	0	0
0.00	*	0	0
0.99	*	0	0
1.00	*	0	0
		sum	0
		activation	0.5

Training a Network



Training pattern:

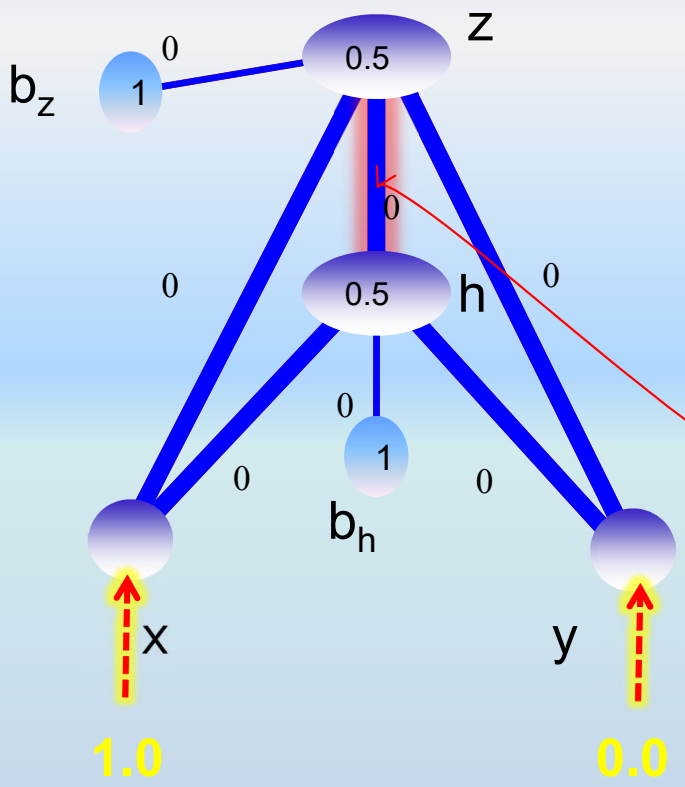
x = 1.0 and **y** = 0.0 , **desired output** = 1.0

Calculate the error on the output unit.

$$\partial_z = (t_z - o_z) * o_z * (1 - o_z)$$

$$\partial_z = (1 - 0.5) * 0.5 * (1 - 0.5) = 0.125$$

Training a Network



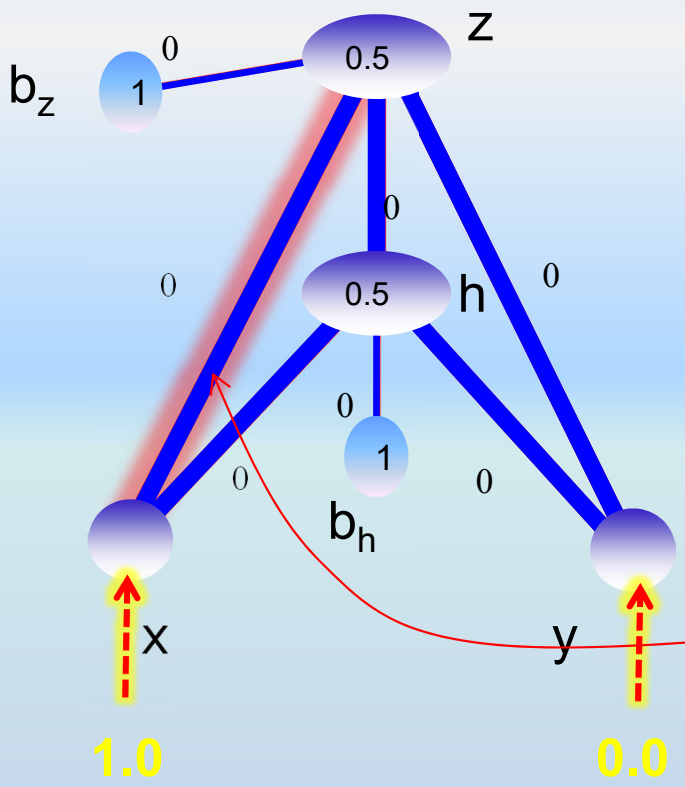
$$\partial_z = 0.125$$

Back-propagate the error to adjust all the weights leading into the output unit.

$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h)$$

Learning rate

Training a Network



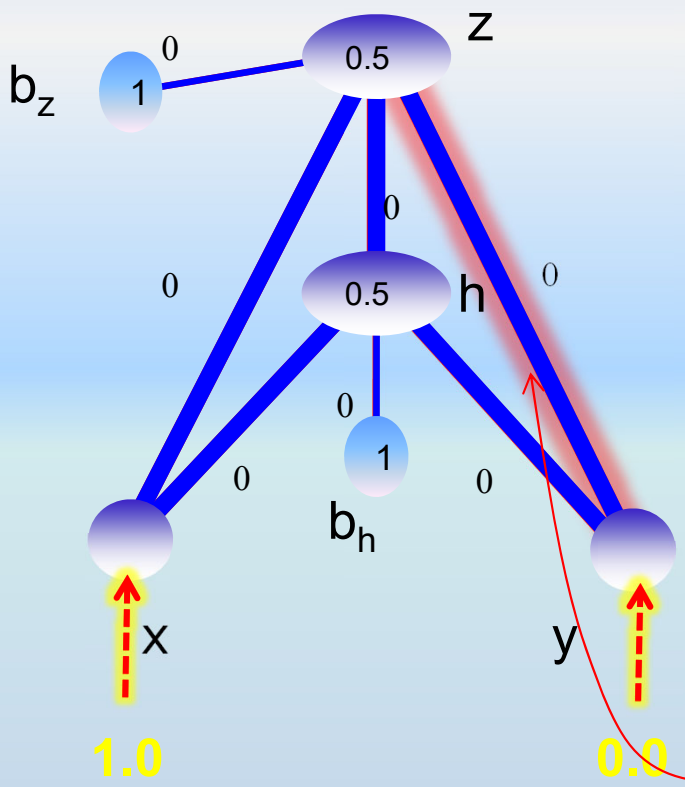
$$\partial_z = 0.125$$

Back-propagate the error to adjust all the weights leading into the output unit.

$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h)$$

$$w_{xz} = w_{xz} + (\eta * \partial_z * o_x)$$

Training a Network



$$\partial_z = 0.125$$

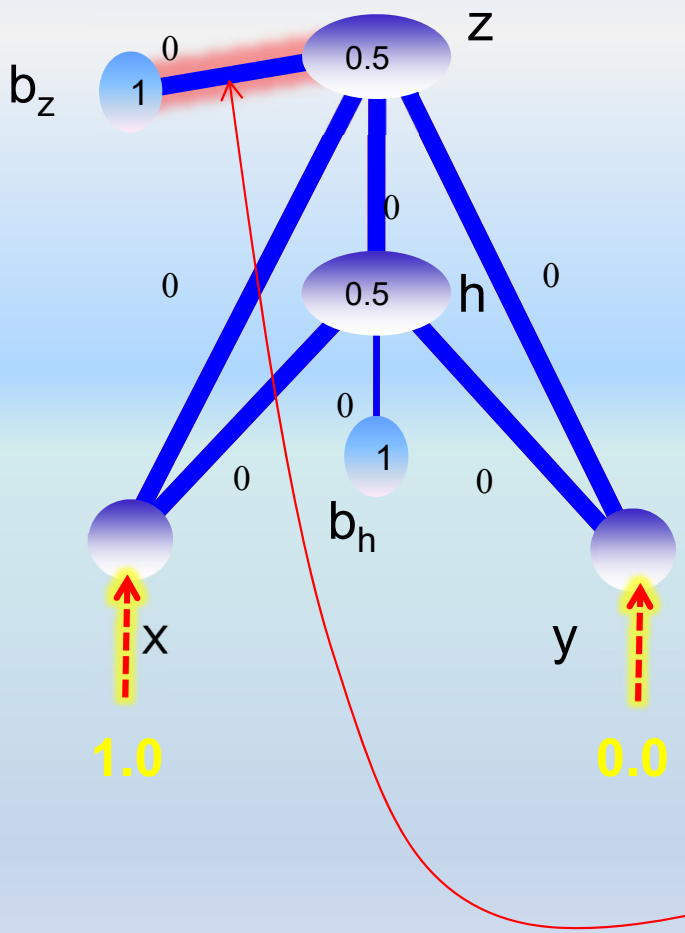
Back-propagate the error to adjust all the weights leading into the output unit.

$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h)$$

$$w_{xz} = w_{xz} + (\eta * \partial_z * o_x)$$

$$w_{yz} = w_{yz} + (\eta * \partial_z * o_y)$$

Training a Network



$$\partial_z = 0.125$$

Back-propagate the error to adjust all the weights leading into the output unit.

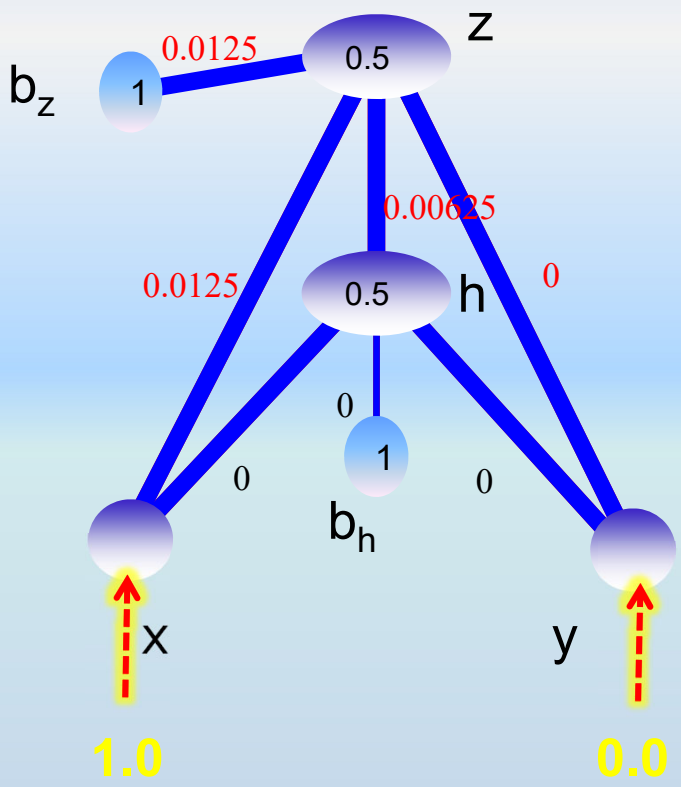
$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h)$$

$$w_{xz} = w_{xz} + (\eta * \partial_z * o_x)$$

$$w_{yz} = w_{yz} + (\eta * \partial_z * o_y)$$

$$w_{b_z z} = w_{b_z z} + (\eta * \partial_z * o_{b_z})$$

Training a Network



$$\partial_z = 0.125$$

Back-propagate the error to adjust all the weights leading into the output unit.

If we set $\eta = 0.1$, we will get the following values:

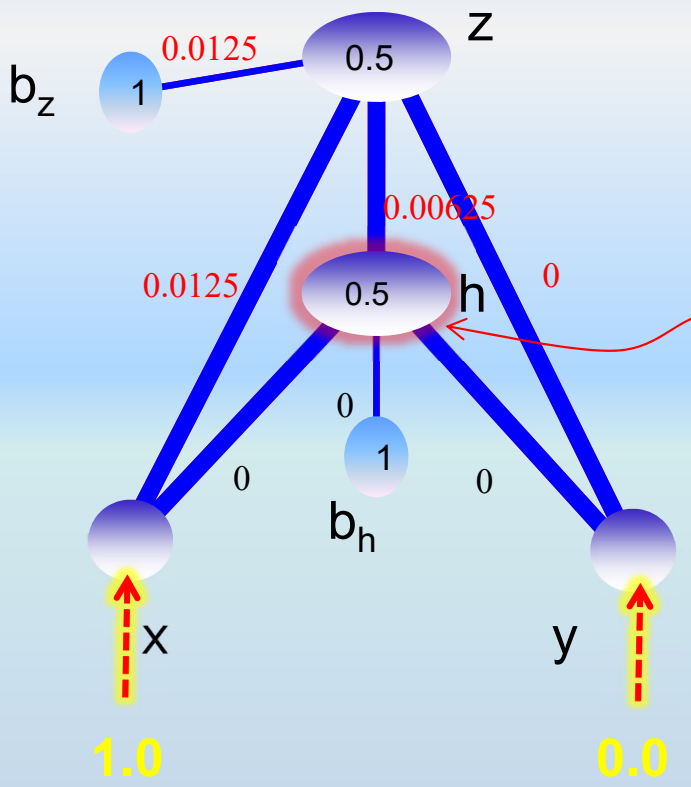
$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h) = 0.00625$$

$$w_{xz} = w_{xz} + (\eta * \partial_z * o_x) = 0.0125$$

$$w_{yz} = w_{yz} + (\eta * \partial_z * o_y) = 0.0$$

$$w_{b_z z} = w_{b_z z} + (\eta * \partial_z * o_{b_z}) = 0.0125$$

Training a Network



Calculate error on the hidden unit ***h***.

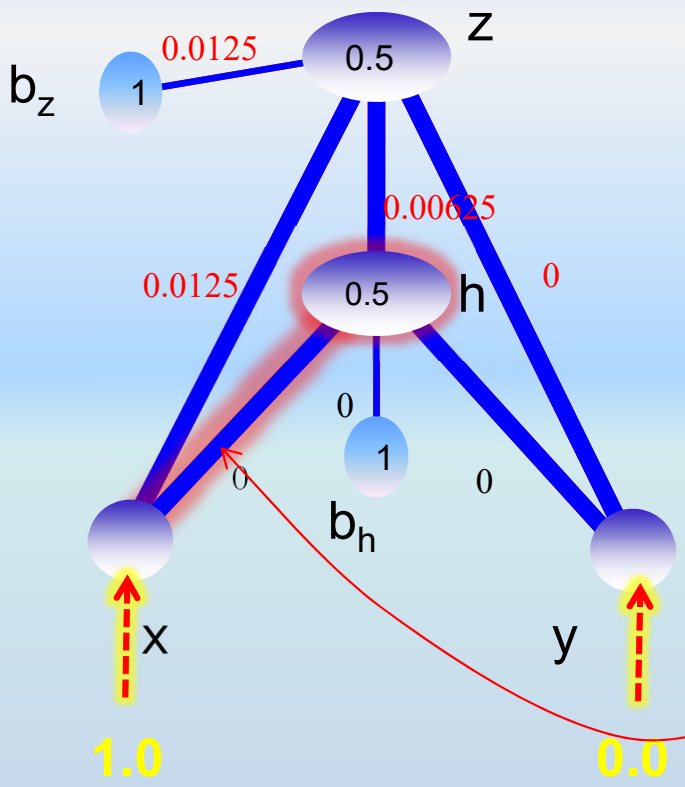
$$\partial_h = o_h * (1 - o_h) \sum_{k=1}^{m} (\partial_k * W_{hk})$$

$$\partial_h = o_h * (1 - o_h) * (\partial_z * W_{hz})$$

The **k** subscript is for all the units in the output layer that are connected directly to node **h**; however, in this example there is only one unit.

$$\partial_h = 0.5 * (1 - 0.5) * 0.125 * 0.00625 = 0.000195$$

Training a Network



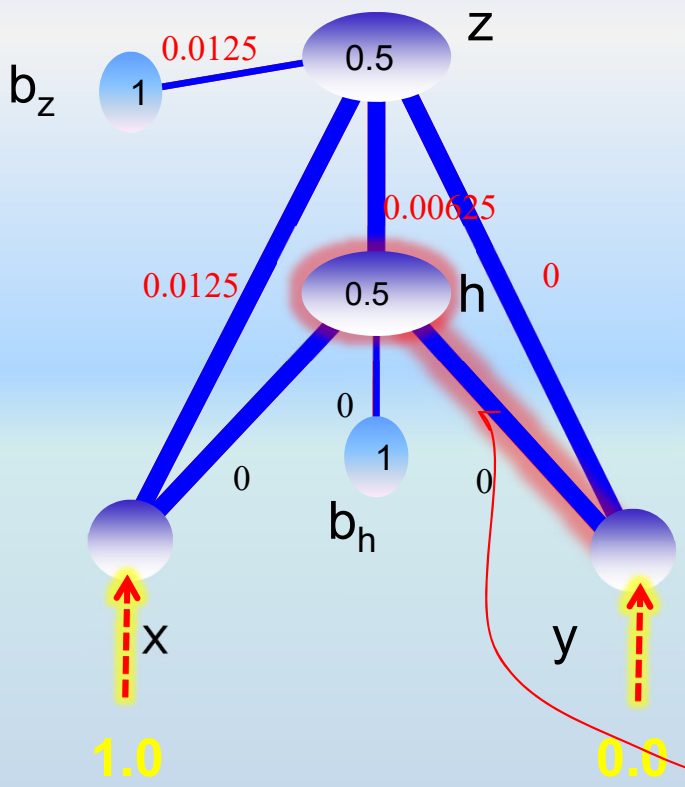
Calculate error on the hidden unit h.

$$\partial_h = o_h * (1 - o_h) * (\partial_z * W_{hz})$$

Back-propagate the error to adjust all the weights leading into the hidden unit.

$$w_{xh} = w_{xh} + (\eta * \partial_h * o_x)$$

Training a Network



Calculate error on the hidden unit h.

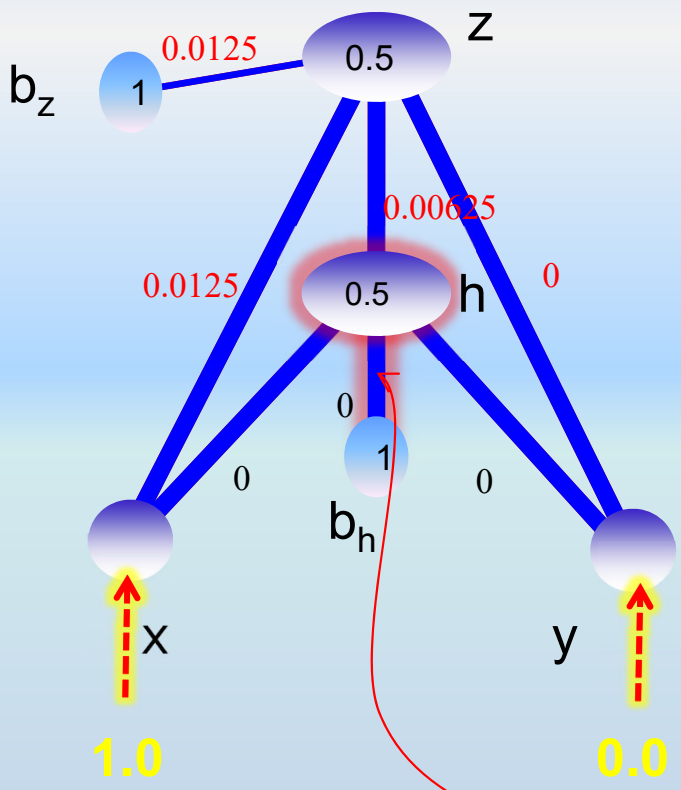
$$\partial_h = (\partial_z * W_{hz}) * o_h * (1 - o_h)$$

Back-propagate the error to adjust all the weights leading into the hidden unit.

$$w_{xh} = w_{xh} + (\eta * \partial_h * o_x)$$

$$w_{yh} = w_{yh} + (\eta * \partial_h * o_y)$$

Training a Network



Calculate error on the hidden unit h.

$$\partial_h = (\partial_z * W_{hz}) * o_h * (1 - o_h)$$

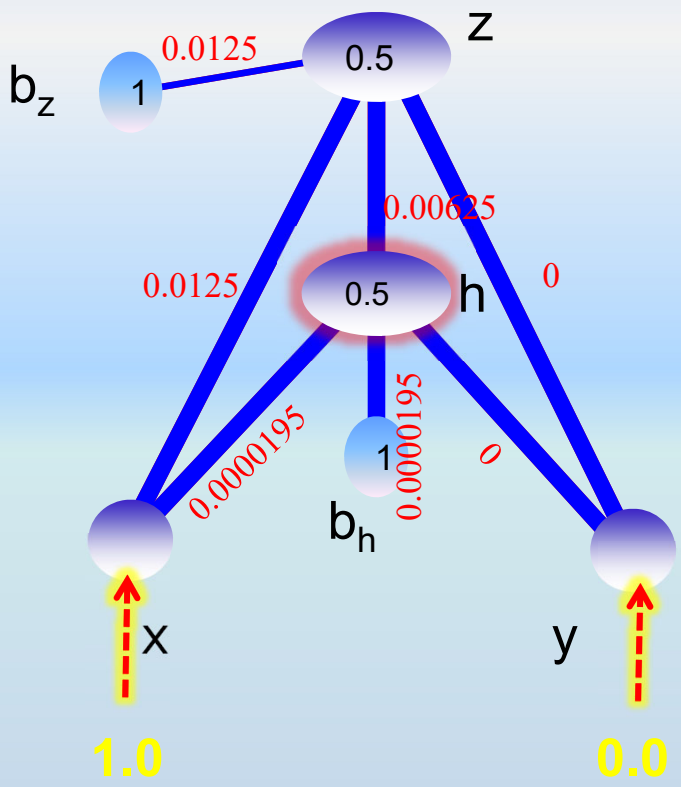
Back-propagate the error to adjust all the weights leading into the hidden unit.

$$w_{xh} = w_{xh} + (\eta * \partial_h * o_x)$$

$$w_{yh} = w_{yh} + (\eta * \partial_h * o_y)$$

$$w_{b_h h} = w_{b_h h} + (\eta * \partial_h * o_{b_h})$$

Training a Network



Calculate error on the hidden unit h.

$$\partial_h = (\partial_z * W_{hz}) * o_h * (1 - o_h) = 0.000195$$

Back-propagate the error to adjust all the weights leading into the hidden unit.

$$w_{xh} = w_{xh} + (\eta * \partial_h * o_x) = 0.0000195$$

$$w_{yh} = w_{yh} + (\eta * \partial_h * o_y) = 0$$

$$w_{b_h h} = w_{b_h h} + (\eta * \partial_h * o_{b_h}) = 0.0000195$$

Backpropagation (Gradient Ascent)

Summary of Weight Update Formulas

$$\frac{dP}{do} = (d - o)$$

$$\Delta w \ \alpha \ \frac{\partial P}{\partial w}$$

$$w = w + \eta \frac{\partial P}{\partial w}$$

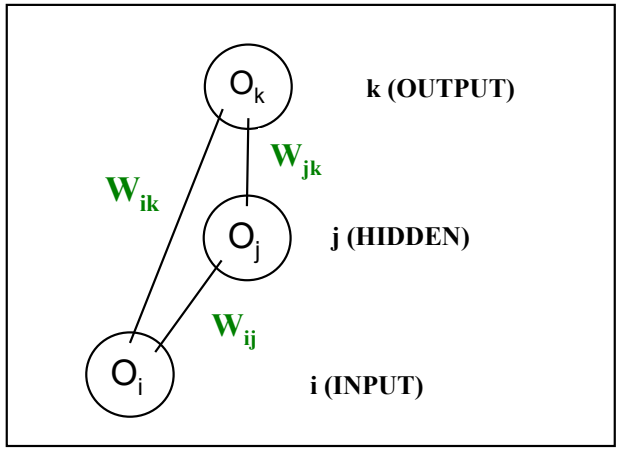
To adjust weights leading into an output unit, W_{jk}:

$$\delta_k = (t_k - O_k) Ok (1 - O_k)$$

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

where:

$$W_{ik} = W_{ik} + \eta \delta_k O_i$$



$\eta - learning_rate$

$\delta_k - error_signal$

$O_j - output_of_unit_j$

To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta \delta_j O_i$$

$\eta - learning_rate$

$\delta_j - error_signal$ at unit j

$O_i - output_of_unit_i$

Training a Network

BACKPROPAGATION TRAINING

After using one training pattern, the activation value for the output layer will now be **0.5070308**. If you will inspect the output of the network on the other three patterns, we have:

x	y	desired	actual
0	0	0	0.5039062
0	1	1	0.5039062
1	0	1	0.5070308
1	1	0	0.5070308

Sad to say but to get the outputs to within **0.1** requires **20,862 *iterations***, a very long time especially for such a short problem. Fortunately, there are a large number of things that can be done to speed-up the training time. The very simplest thing to do is to increase the learning rate, η. The following table shows how many iterations are used for different values of η.

η	iterations
0.1	20,862
0.5	2,455
1.0	1,060
2.0	480
3.0	(fails)



Derivation of Formulas

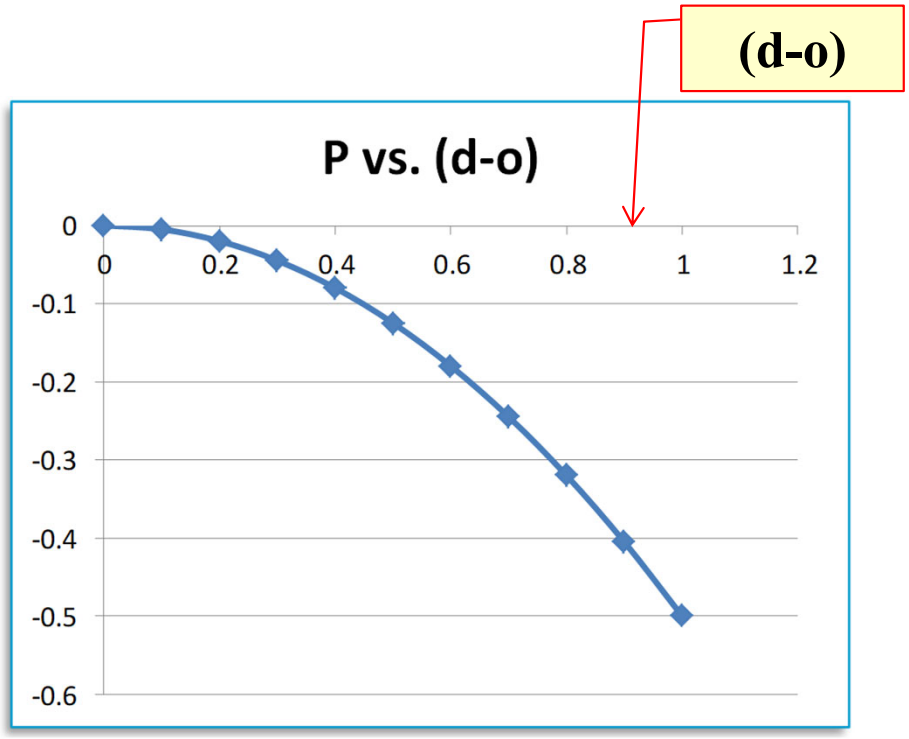
Let’s see how calculus was applied to extract the back propagation formulas…

Gradient Ascent

Performance Function

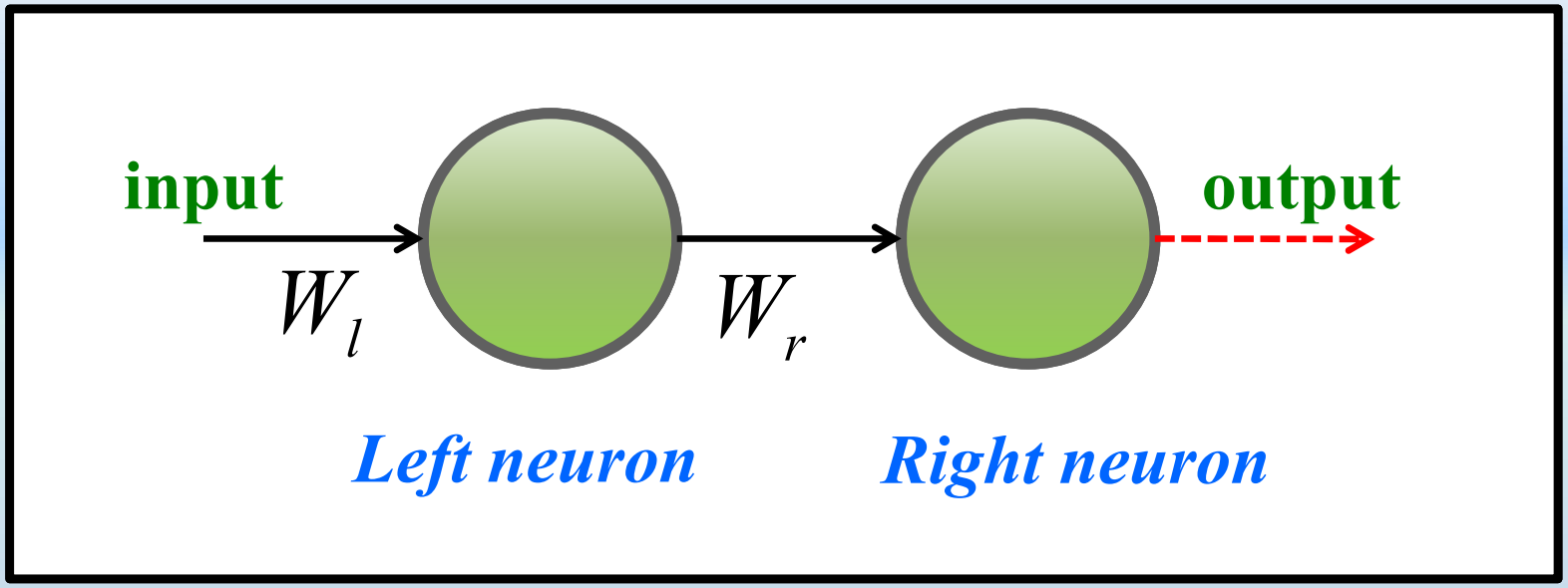
$$P = (-1/2) * ((desired_output - actual_output)^2)$$

desired output	actual output	difference	Performance
1	1	0	0
1	0.9	0.1	-0.005
1	0.8	0.2	-0.02
1	0.7	0.3	-0.045
1	0.6	0.4	-0.08
1	0.5	0.5	-0.125
1	0.4	0.6	-0.18
1	0.3	0.7	-0.245
1	0.2	0.8	-0.32
1	0.1	0.9	-0.405
1	0	1	-0.5



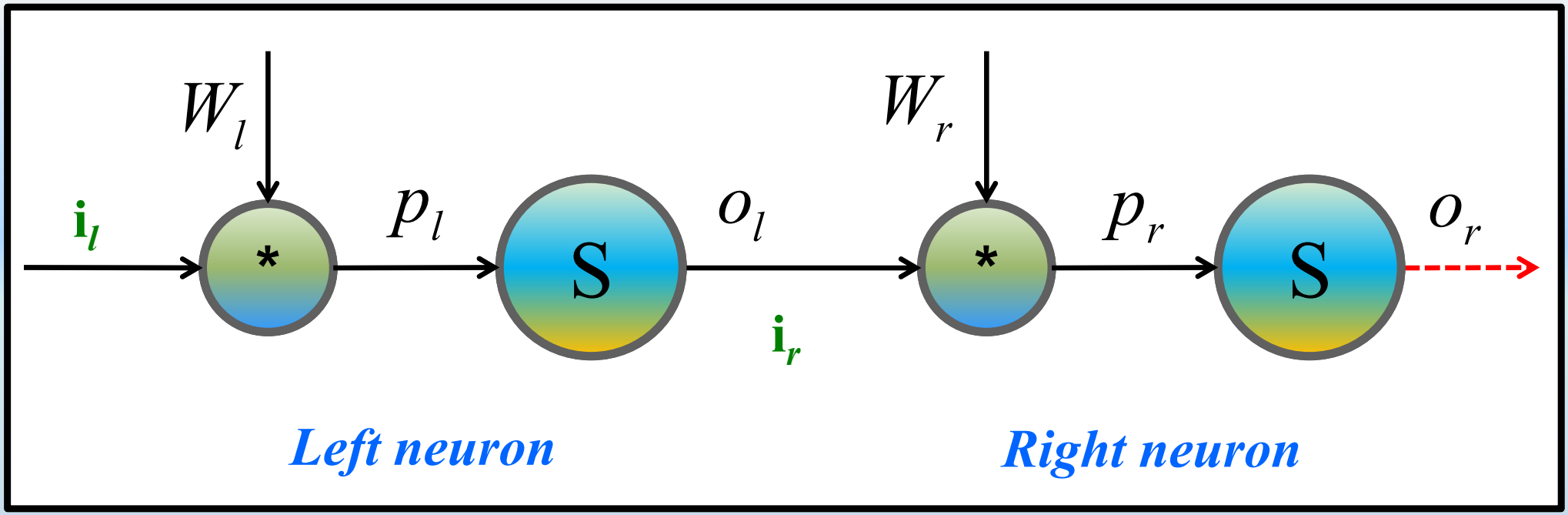
When training the network, we would like to maximise P

Let's consider the simplest possible neural network that will help explain how the back propagation formulas were derived.



2 neurons, 1 input, 1 output

In terms of the processes involved, let's expand the graphical representation of our network of neurons.



See the details of the formula derivations in a separate pdf file (downloadable via Stream).

2 neurons, 1 input, 1 output

Gradient Ascent

Summary of Weight Update Formulas

$$\frac{dP}{do} = (d - o)$$

$$\Delta w \ \alpha \ \frac{\partial \mathrm{P}}{\partial w}$$

$$w = w + \eta \frac{\partial \mathrm{P}}{\partial w}$$

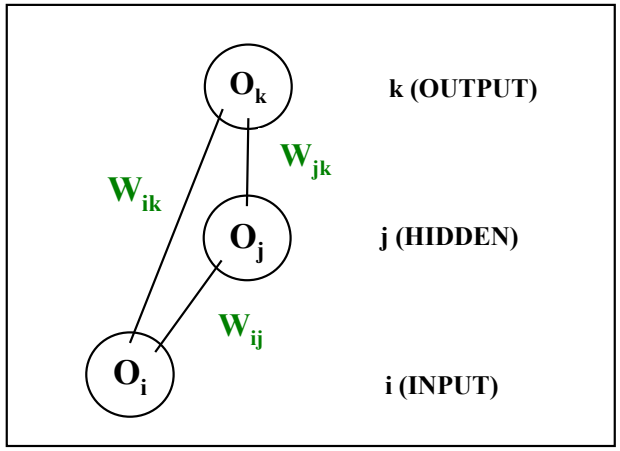
To adjust weights leading into an output unit, W_{jk}:

$$\delta_k = (t_k - O_k) Ok (1 - O_k)$$

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

$$W_{ik} = W_{ik} + \eta \delta_k O_i$$

$\eta - learning_rate$
$\delta_k - error_signal$
$O_j - output_of_unit_j$



To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta \delta_j O_i$$

$\eta - learning_rate$
$\delta_j - error_signal$ at unit j
$O_i - output_of_unit_i$

References

- MIT OpenCourseWare
- www.cim.mcgill.ca/~jer/courses/ai/readings/backprop.pdf
- Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead